Final system:

Section 1: Initial System Configuration

Objective:

Configure the system's UART prescale, parity settings, and clock divider ratio before executing the main test cases.

Executed Commands:

```
write_cmd_sequence(8'h02, 8'b10000001);
write_cmd_sequence(8'h03, 8'h20);
```

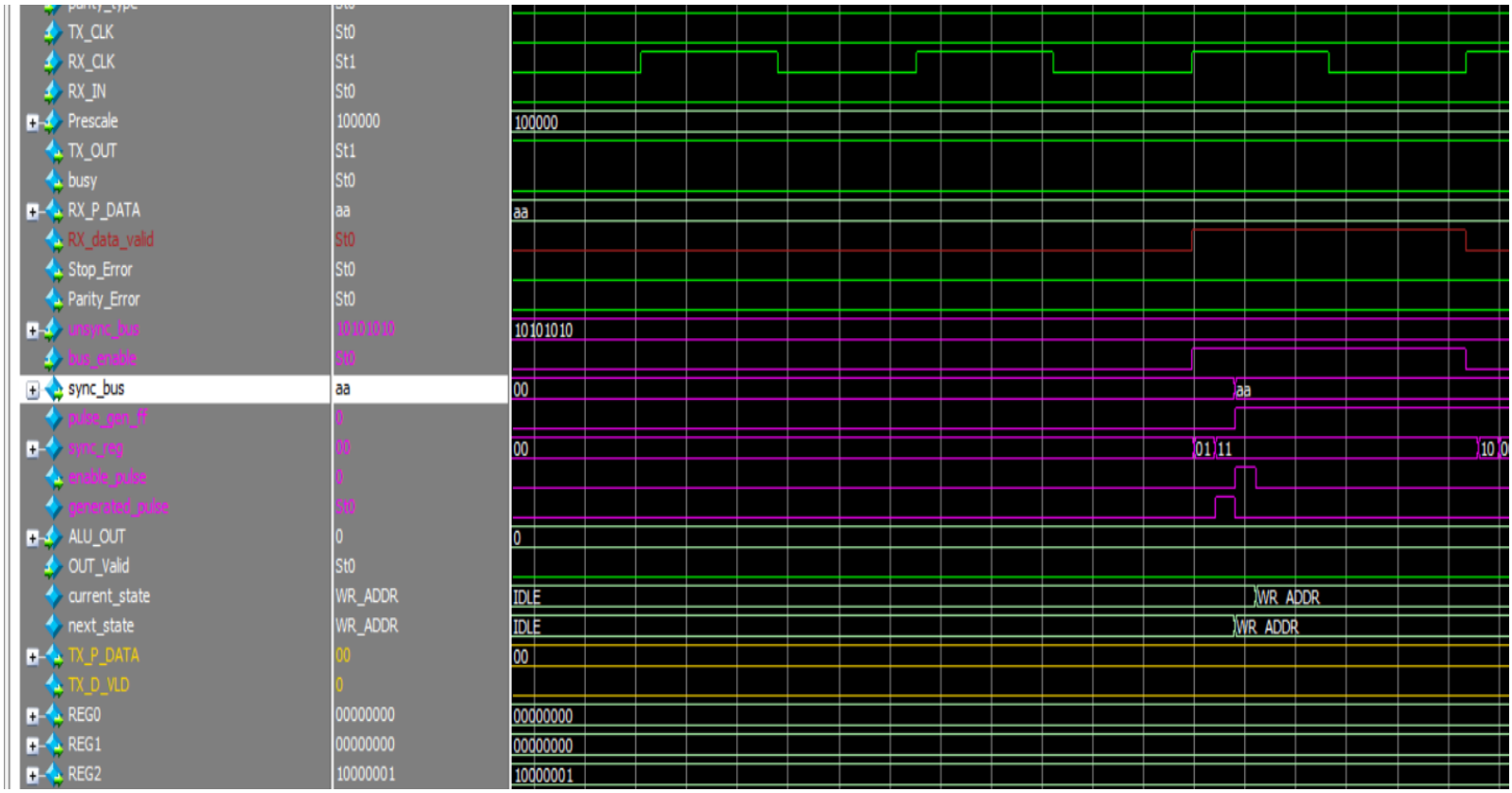
Configuration Details:

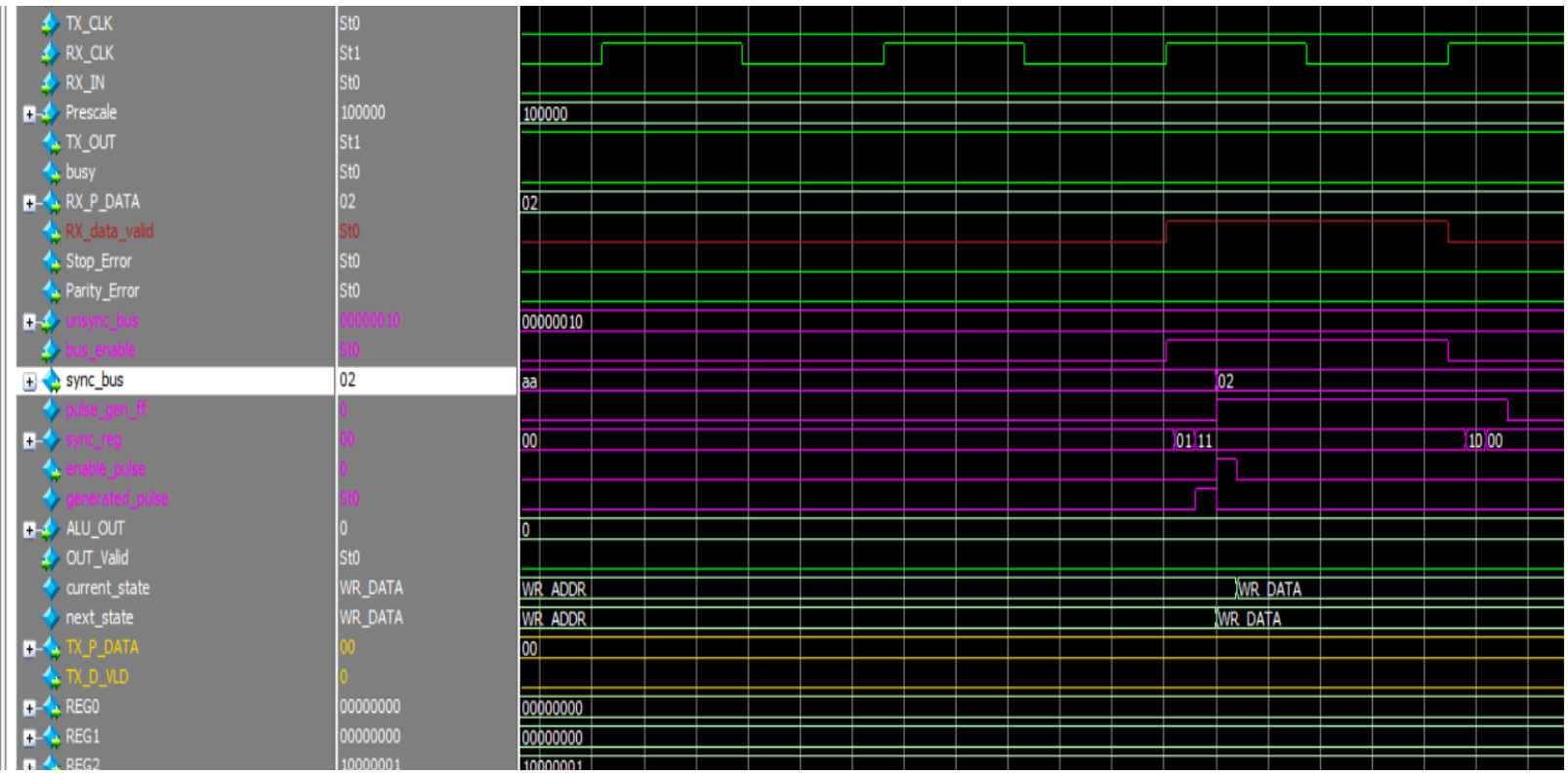
Register	Address	Written Data	Configuration
REG2	0x02	0x81	Prescale = 32, Even Parity Enabled
REG3	0x03	0x20	Clock Division Ratio = 32

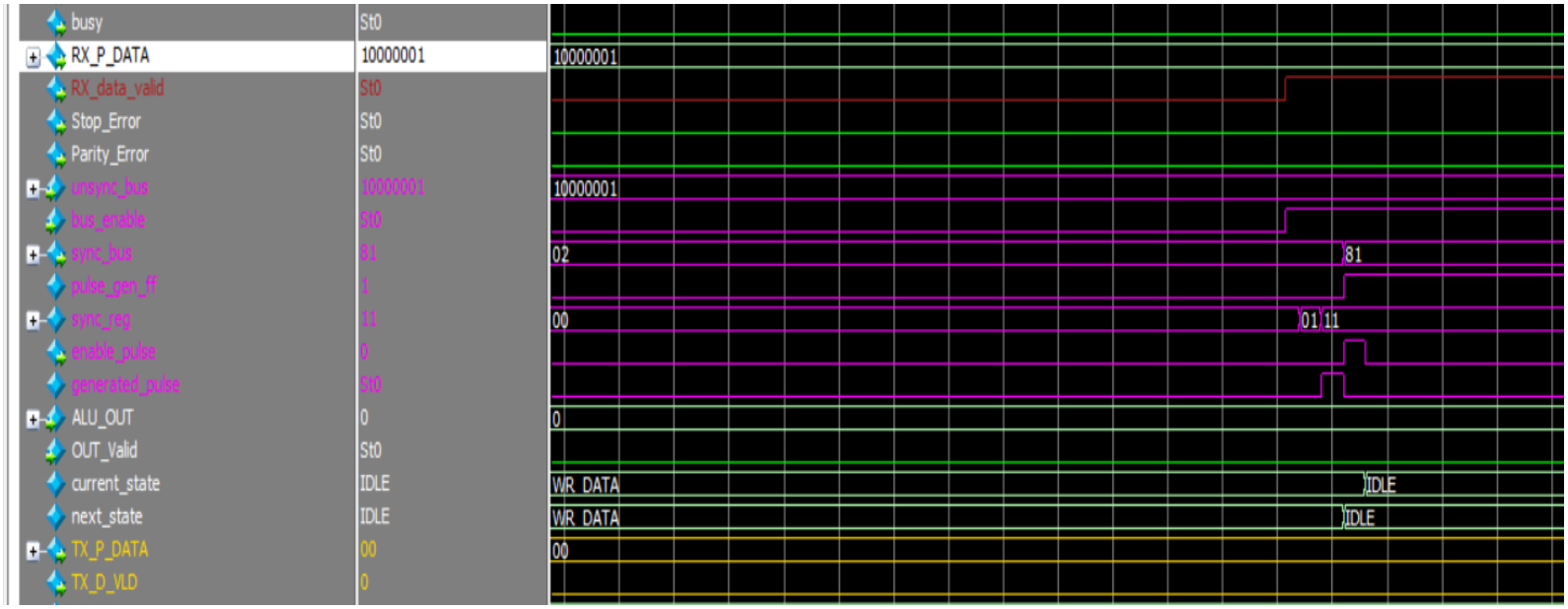
Expected Results:

- REG2 should be updated to `8'b10000001` (0x81).
- REG3 should be updated to `8'b00100000` (0x20).
- The system should complete the configuration successfully without errors.

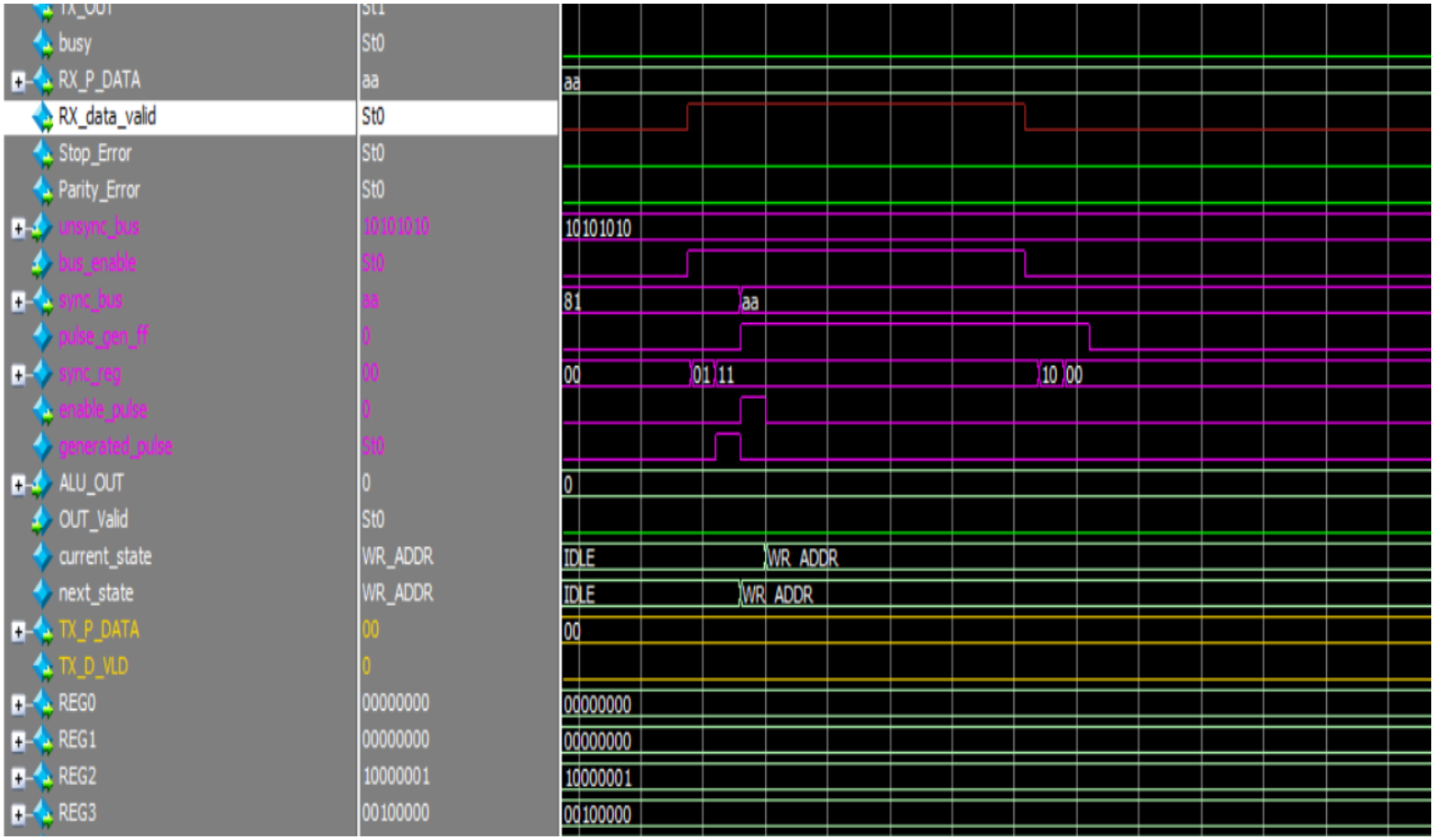
Waveform: write configuration REG2

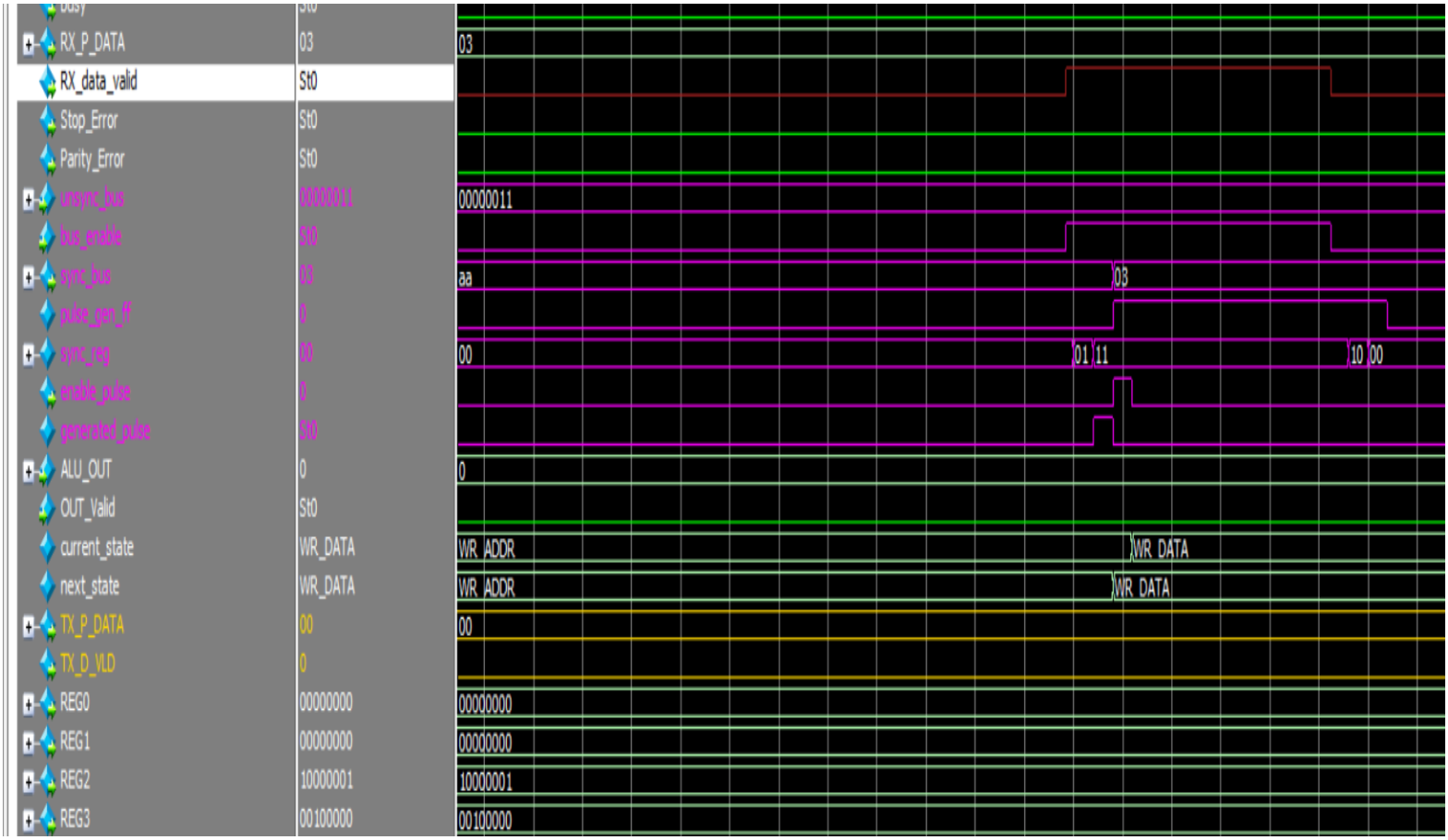


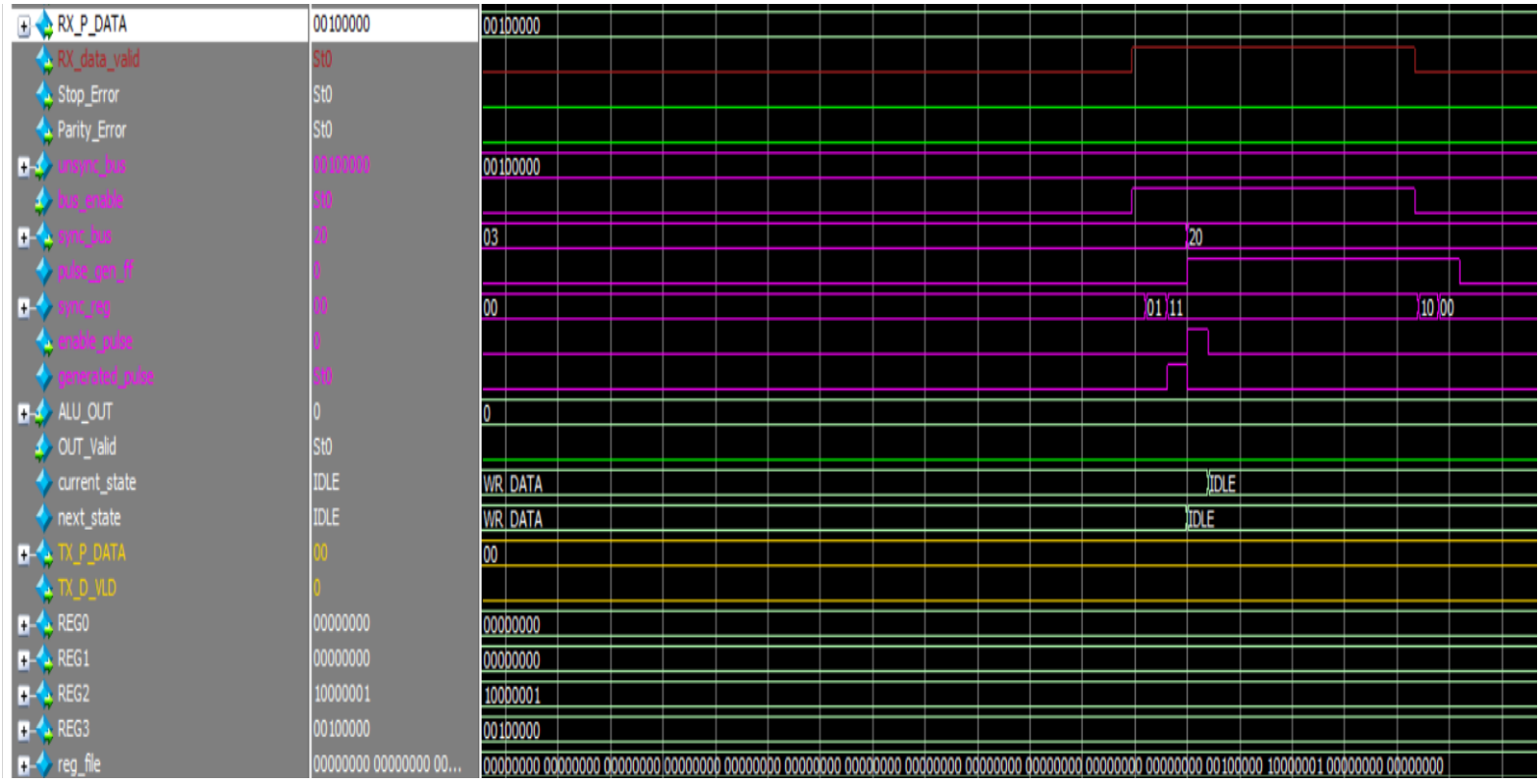




write configuration REG3

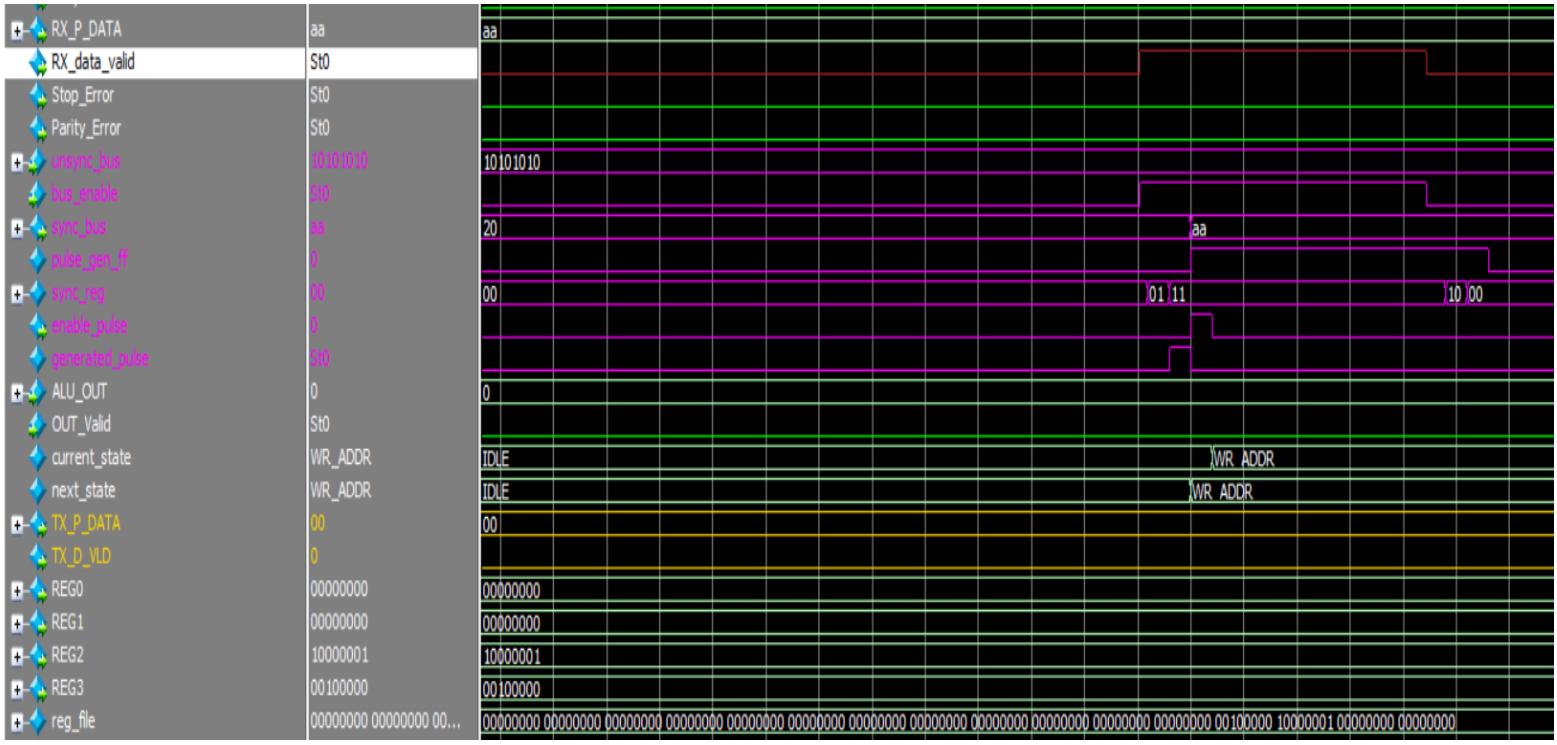


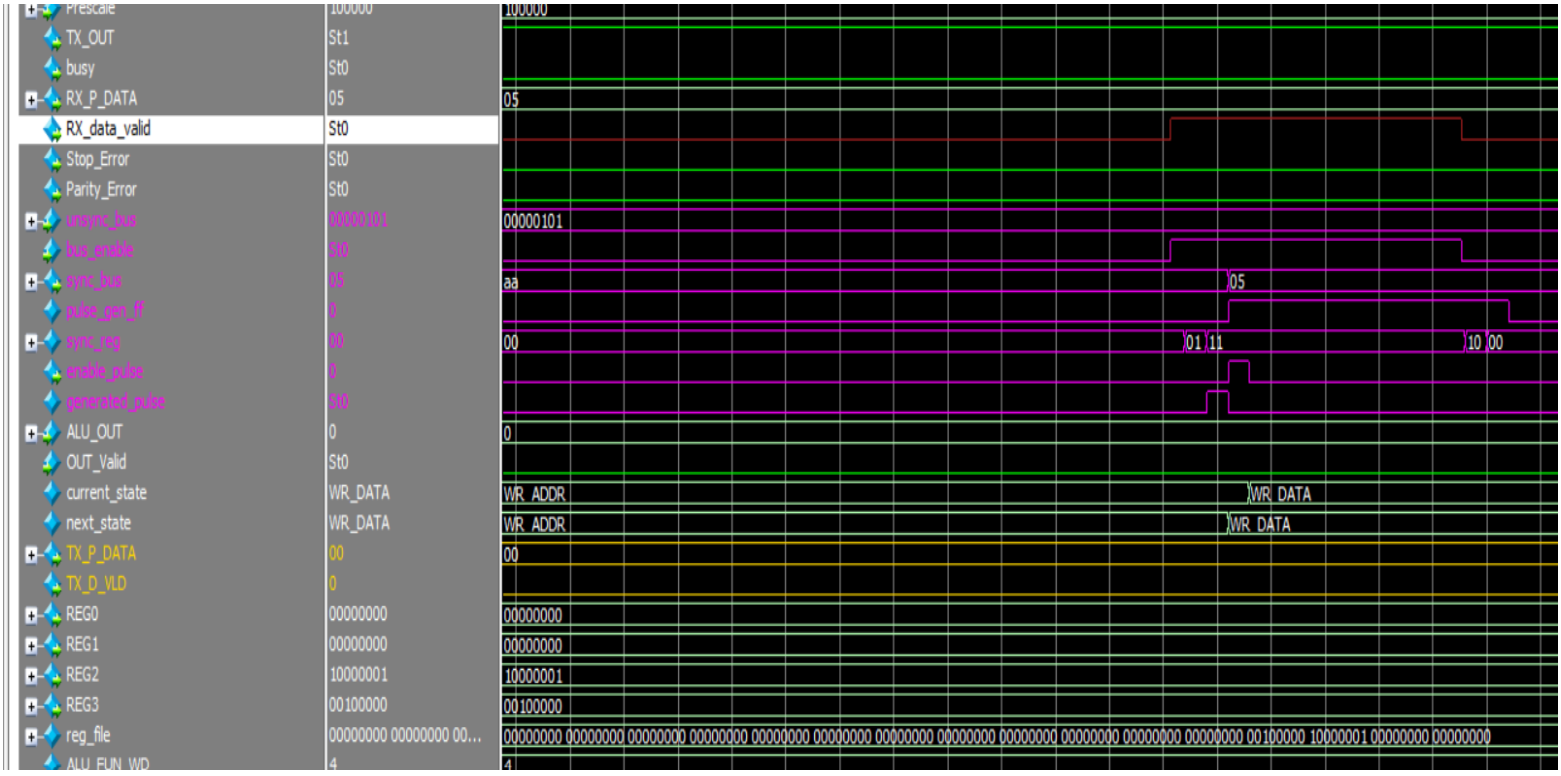

RX_P_DATA
03
RX_data_valid
St0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00000011
bus_enable
St0
sync_bus
03
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
WR_DATA
next_state
WR_DATA
TX_P_DATA
00
TX_D_VLD
0
REG0
00000000
REG1
00000000
REG2
10000001
REG3
00100000
aa
03
01
11
10
WR_ADDR
WR_DATA

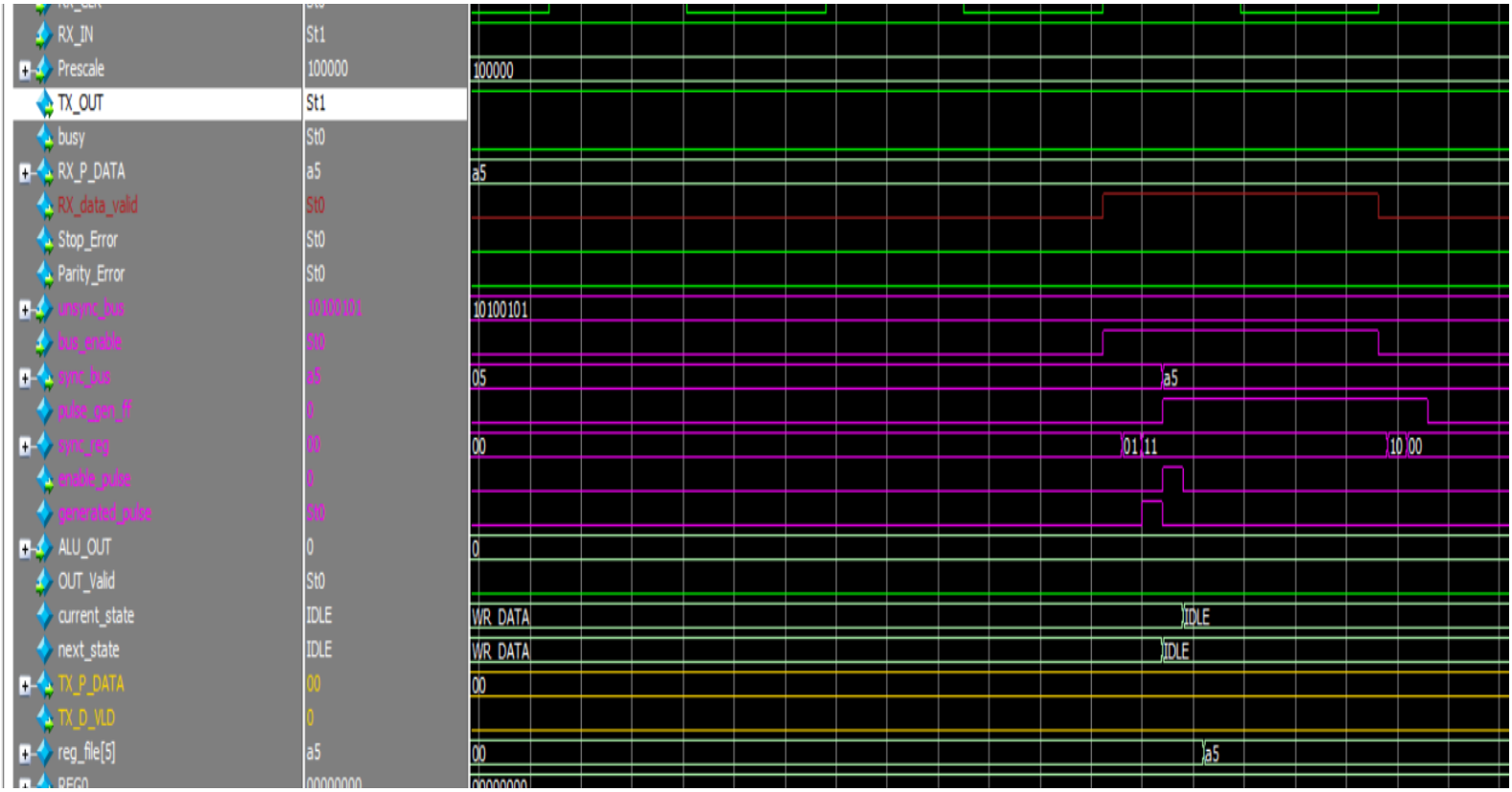

RX_P_DATA
00100000
RX_data_valid
St0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00100000
bus_enable
St0
sync_bus
20
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
IDLE
next_state
IDLE
TX_P_DATA
00
TX_D_VLD
0
REG0
00000000
REG1
00000000
REG2
10000001
REG3
00100000
reg_file
00000000 00000000 00...
03
20
01
11
10
WR_DATA
IDLE

2: Testing RegFile Write & Read Operations

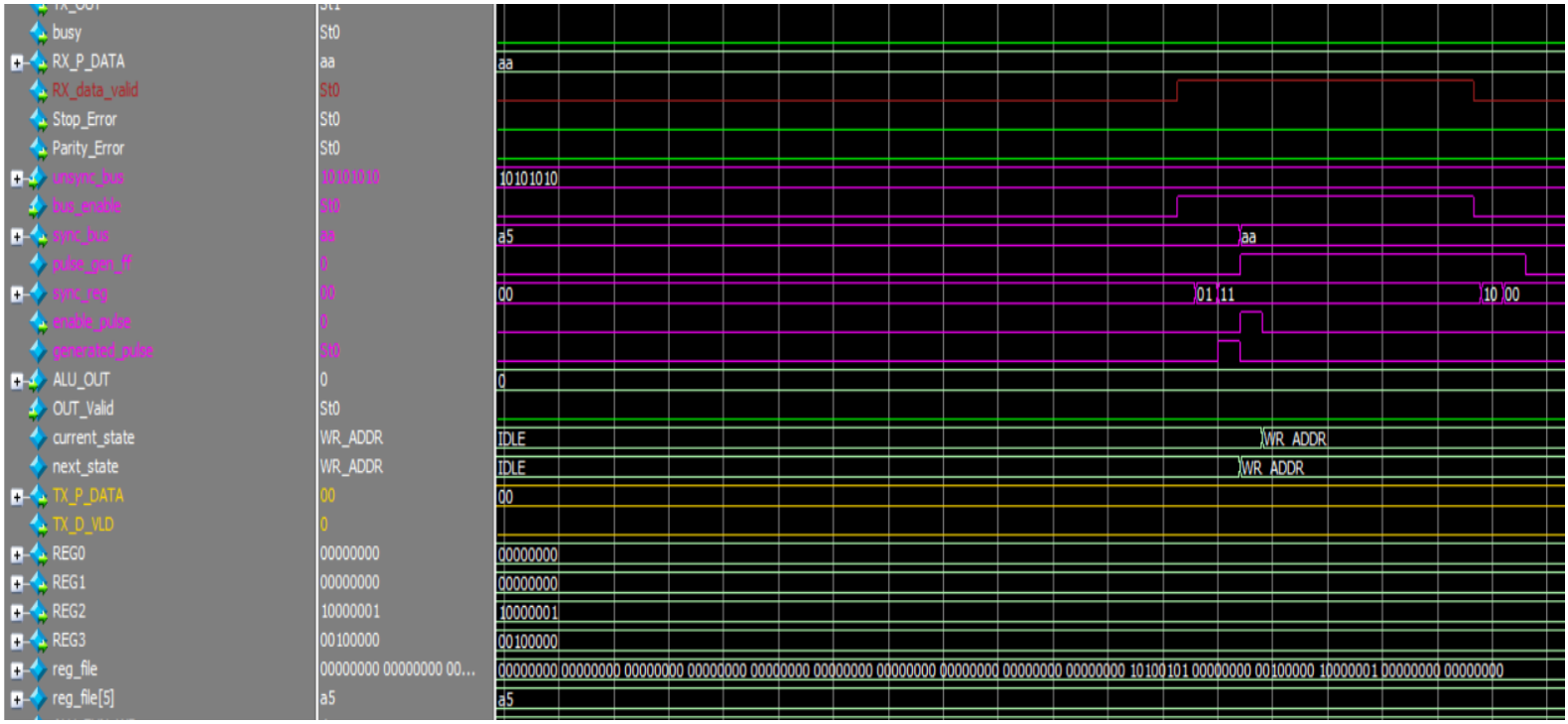
write_cmd_sequence(8'h05, 8'hA5);

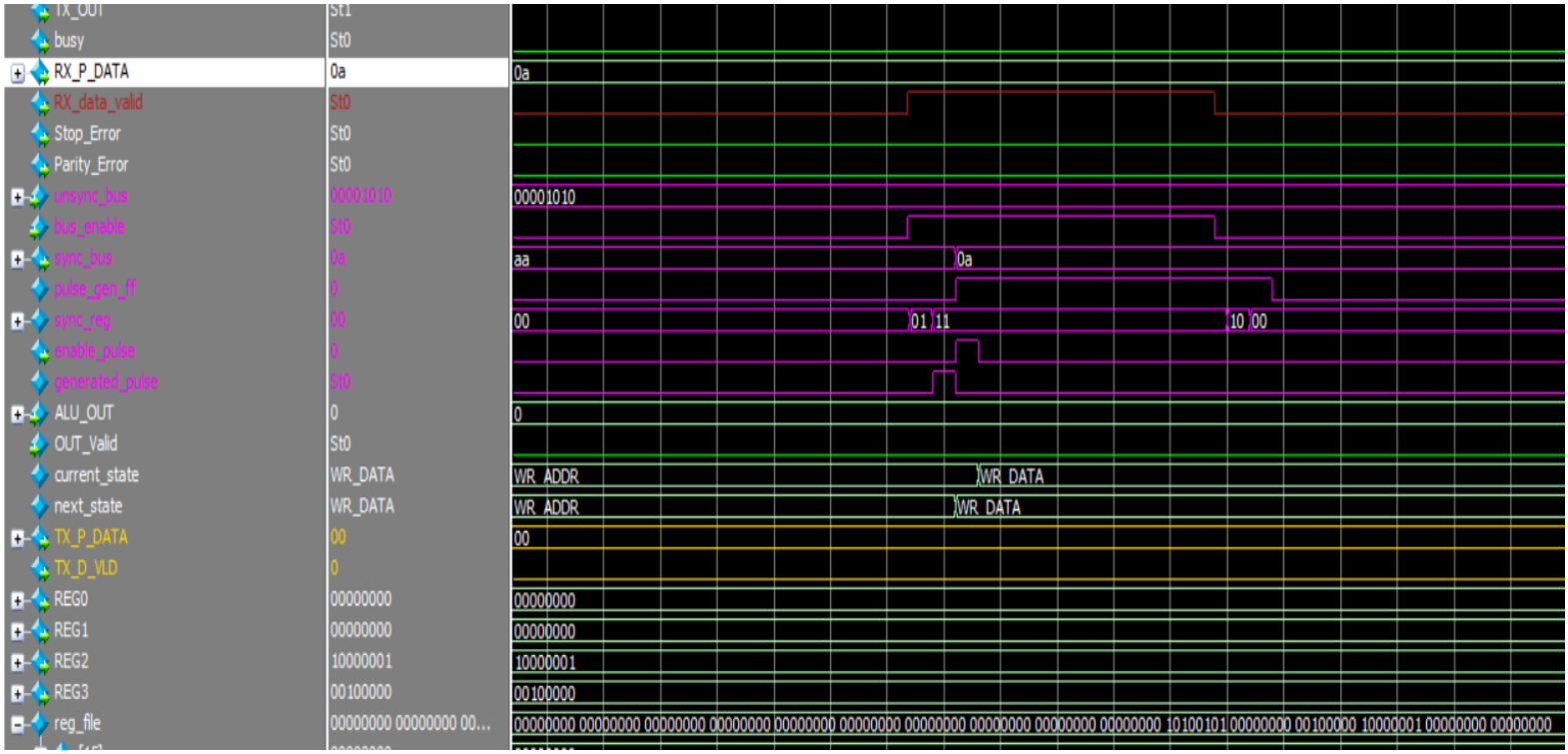


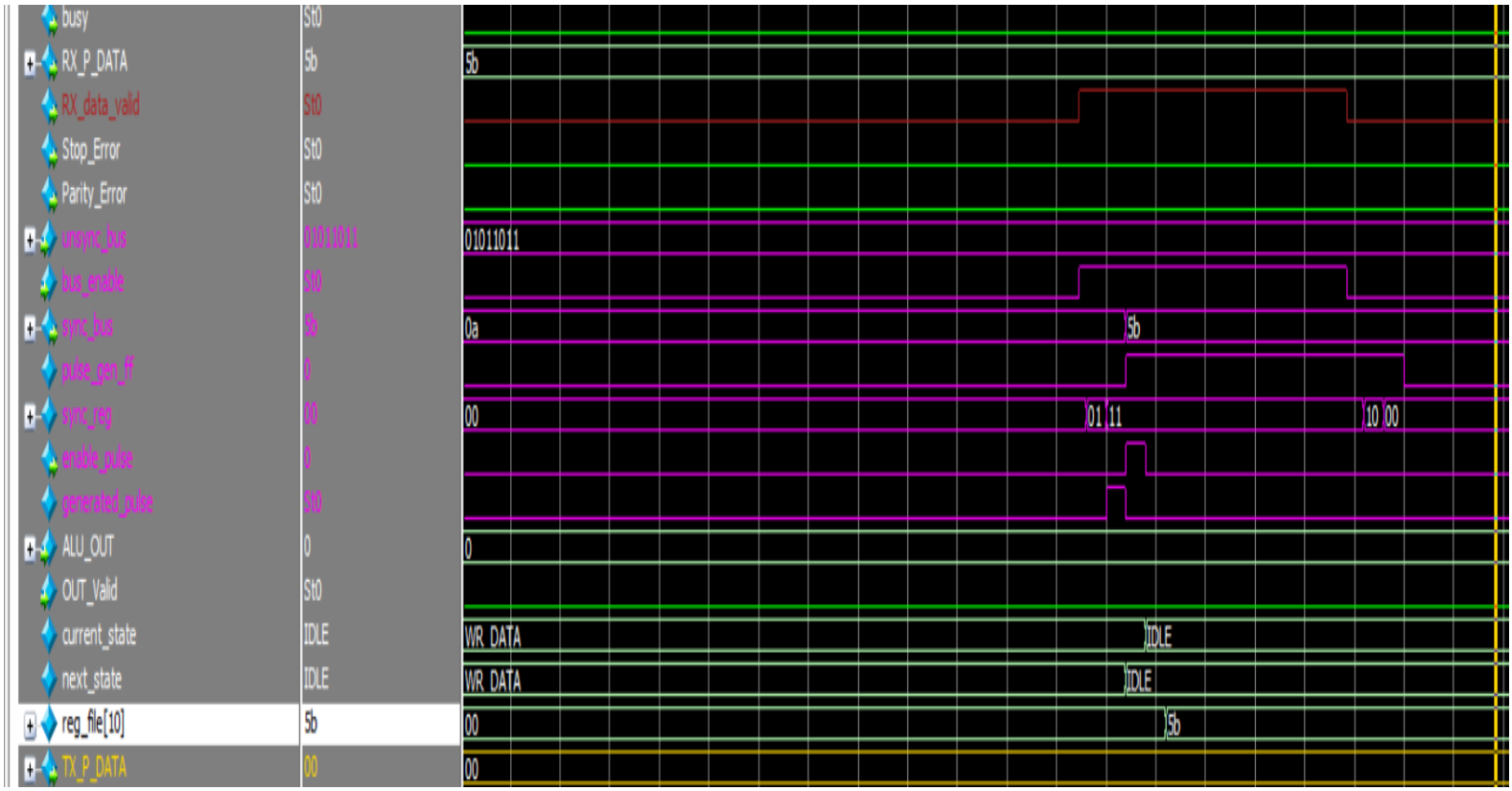




write_cmd_sequence(8'h0A, 8'h5B);

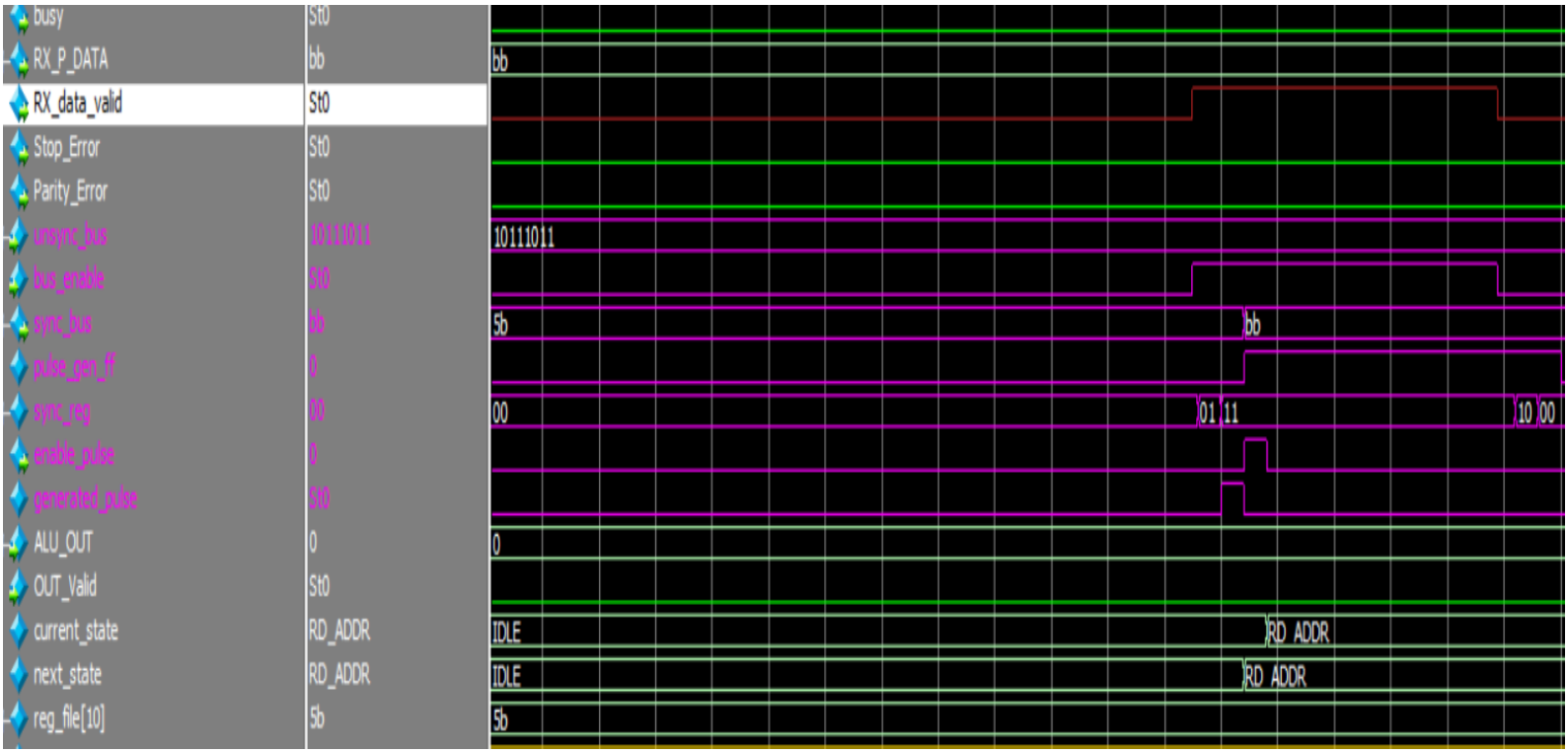


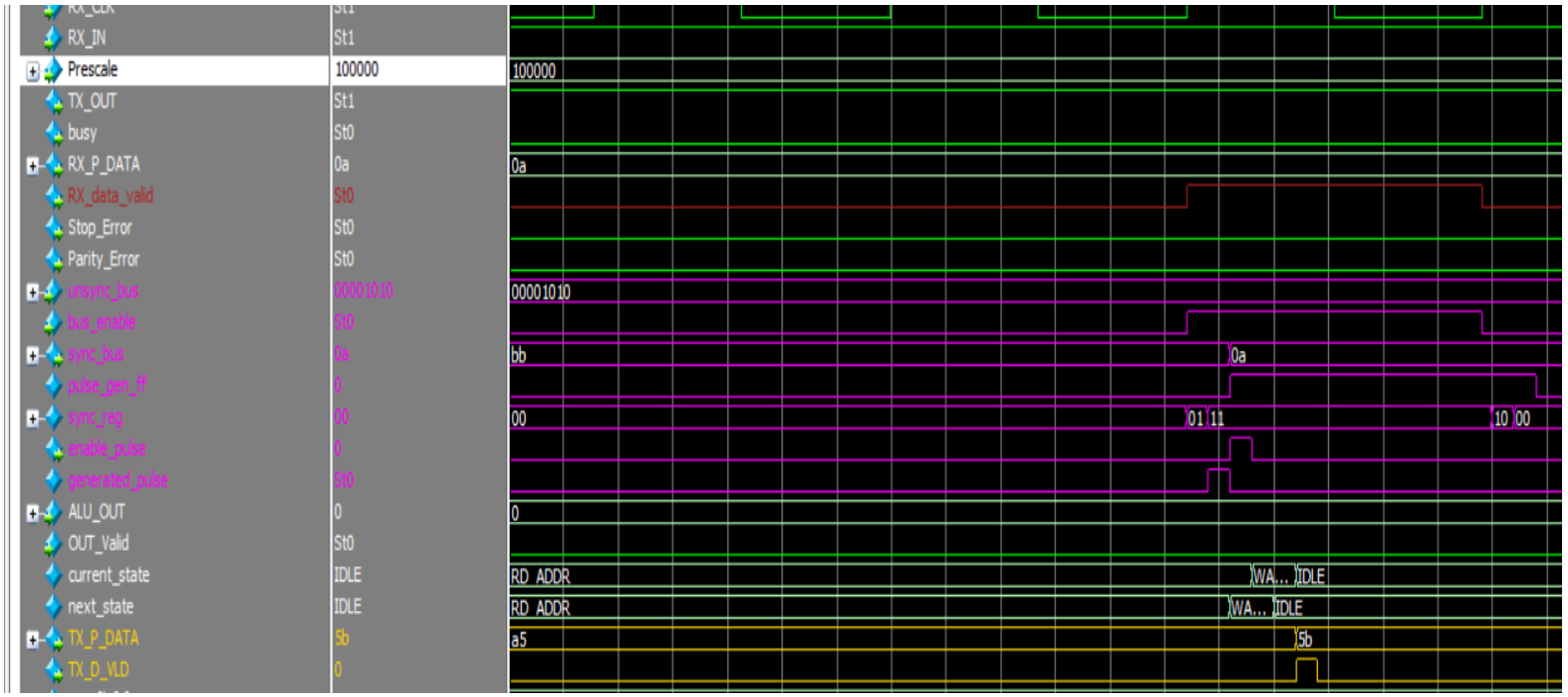

busy
St0
RX_P_DATA
0a
RX_data_valid
St0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00001010
bus_enable
St0
sync_bus
0a
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
WR_DATA
next_state
WR_DATA
TX_P_DATA
00
TX_D_VLD
0
REG0
00000000
REG1
00000000
REG2
10000001
REG3
00100000
reg_file
00000000 00000000 00...
aa
0a
00
01
11
10
00
WR_ADDR
WR_DATA
WR_ADDR
WR_DATA
00
0
00000000
00000000
10000001
00100000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 10100101 00000000 00100000 10000001 00000000 00000000


busy
St0
RX_P_DATA
5b
RX_data_valid
St0
Stop_Error
St0
Parity_Error
St0
unsync_bus
01011011
bus_enable
St0
sync_bus
5b
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
IDLE
next_state
IDLE
reg_file[10]
5b
TX_P_DATA
00
01011011
0a
5b
00
01
11
10
00
0
WR_DATA
IDLE
WR_DATA
IDLE
00
5b
00

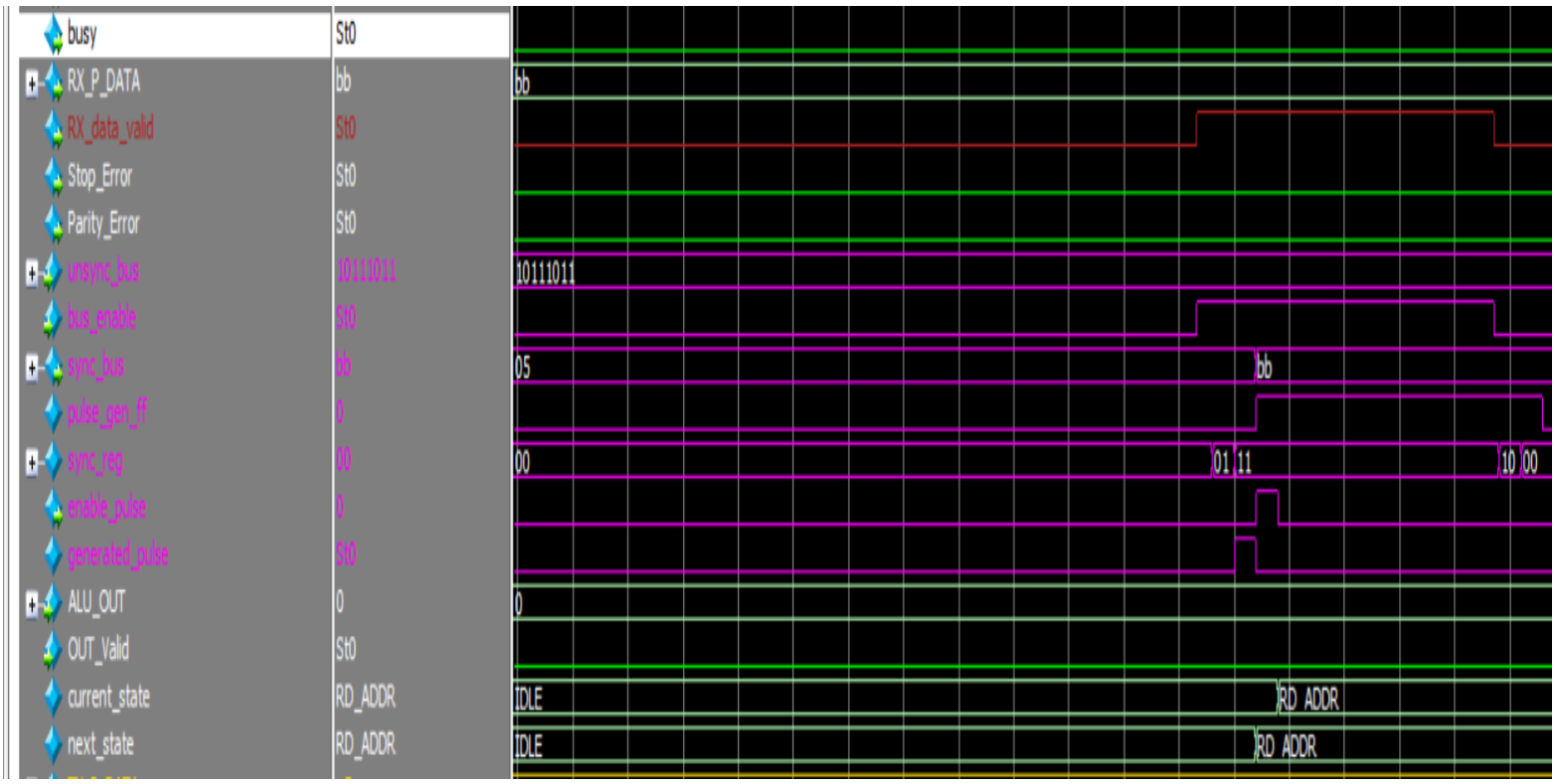
READ OPERATION:

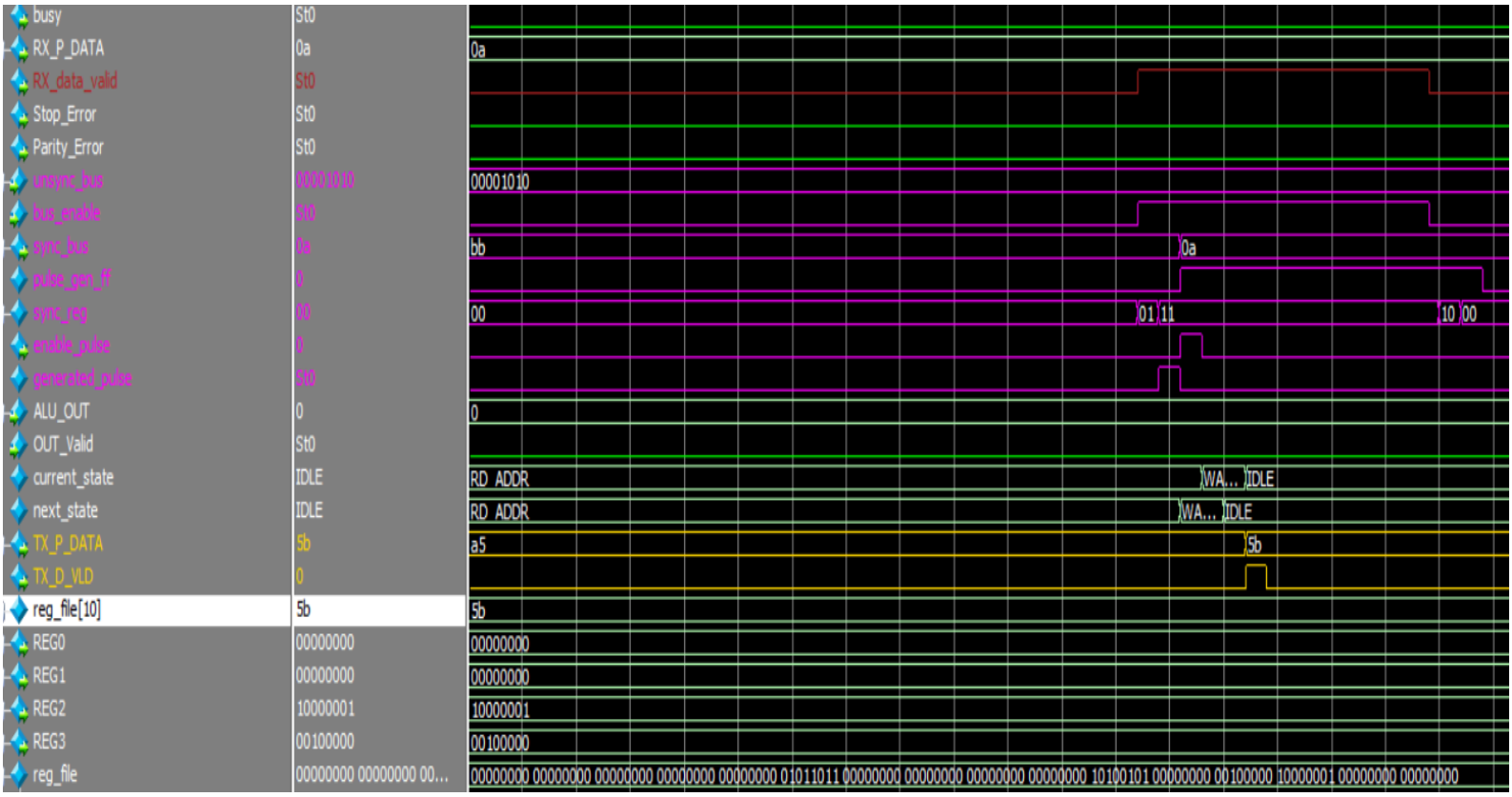
read_and_check_cmd(8'h05, 8'hA5);





read_and_check_cmd(8'h0A, 8'h5B);

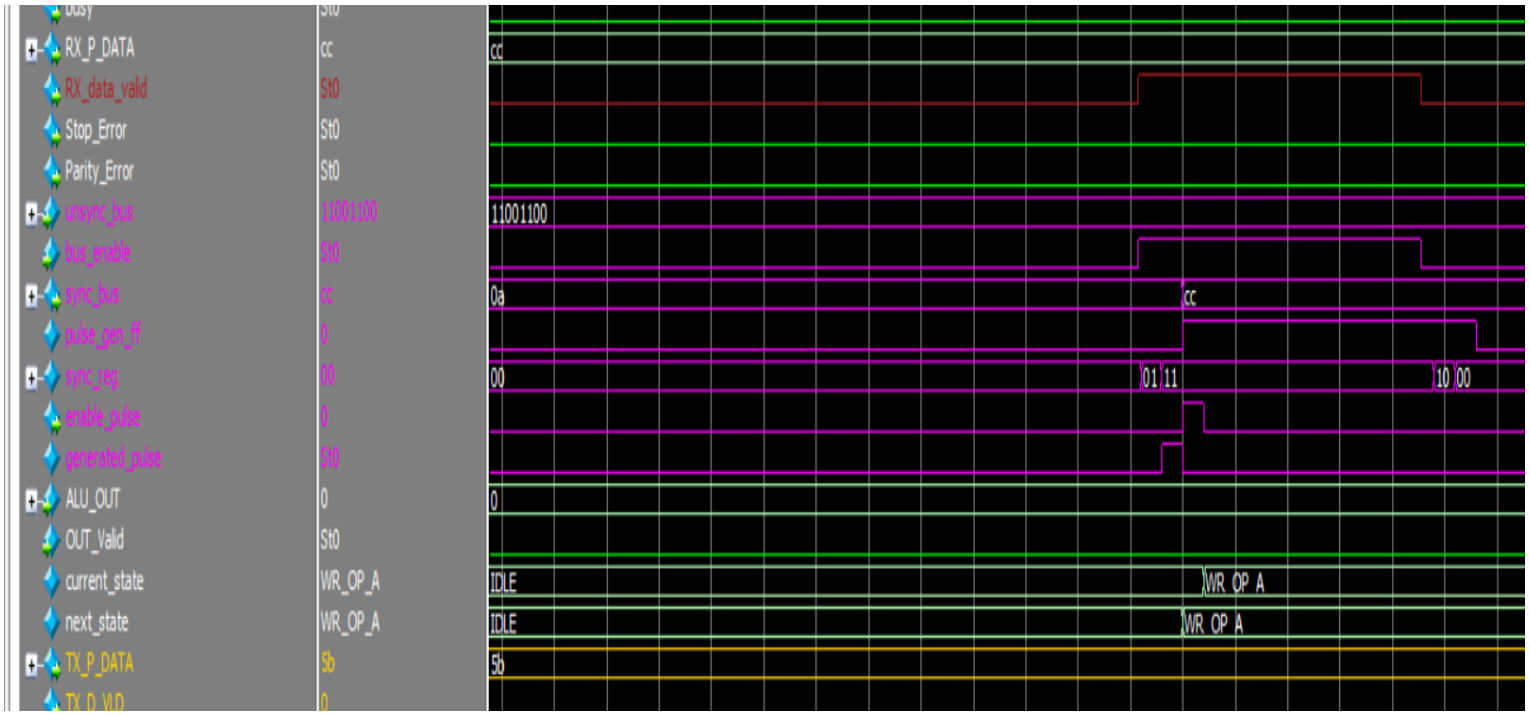


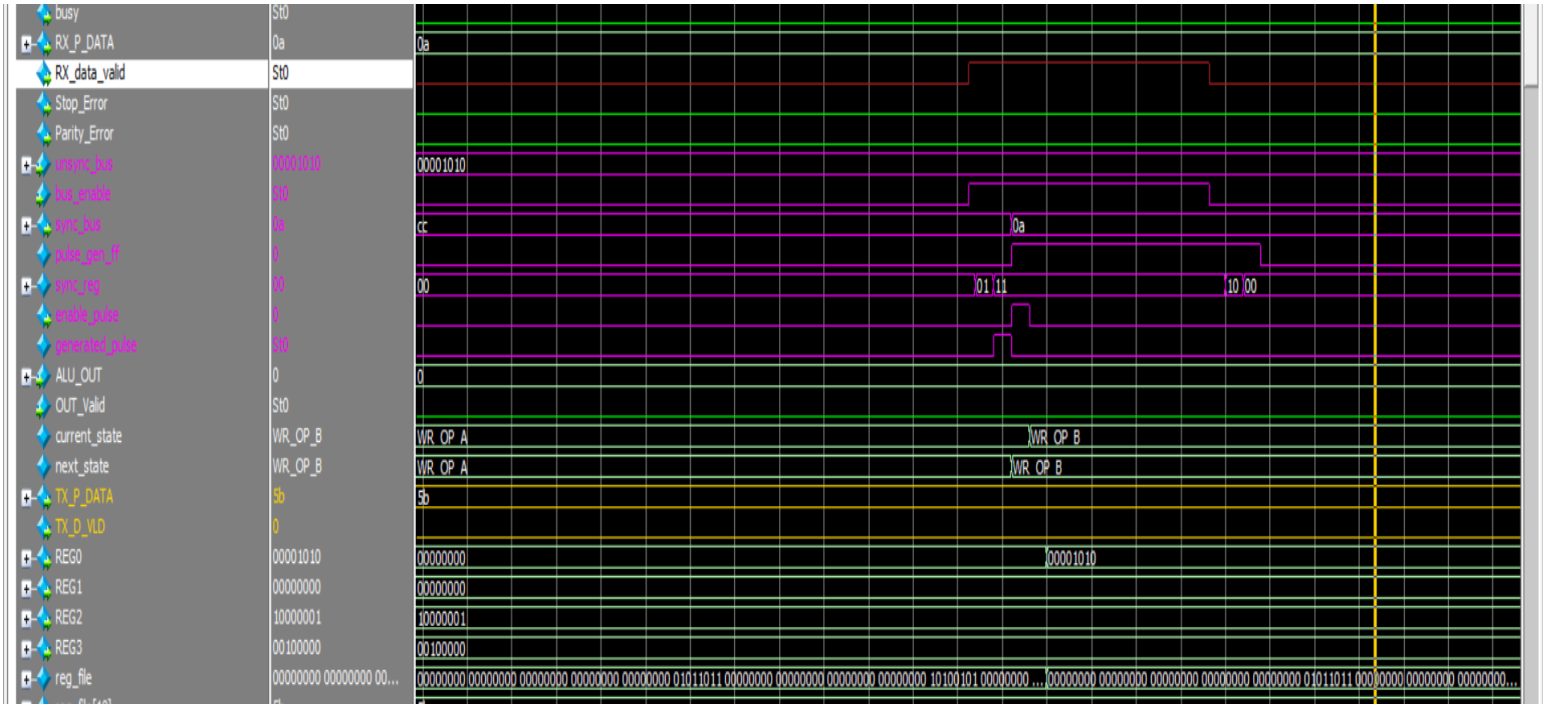


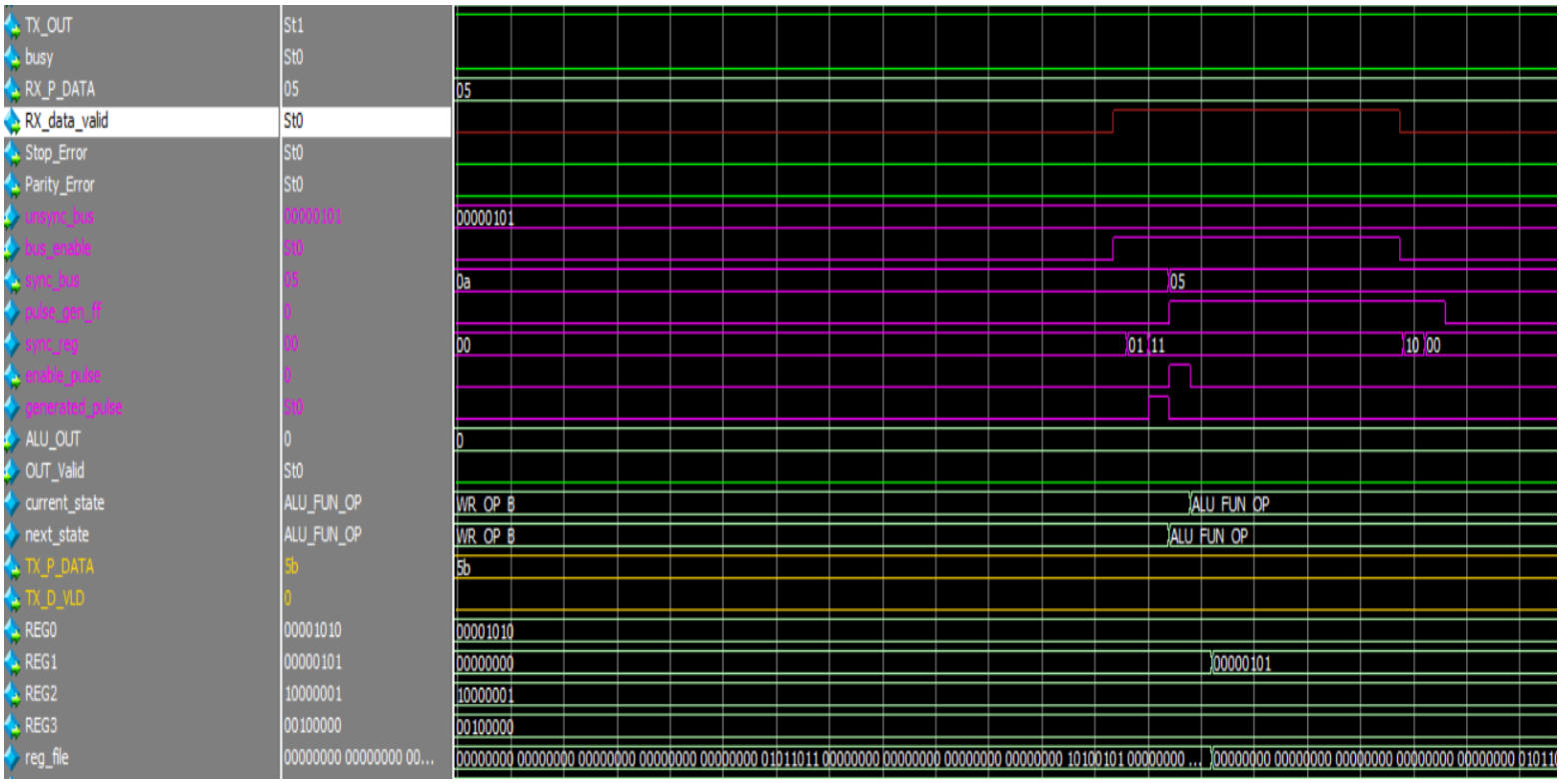
3: Testing ALU Operations with Operands

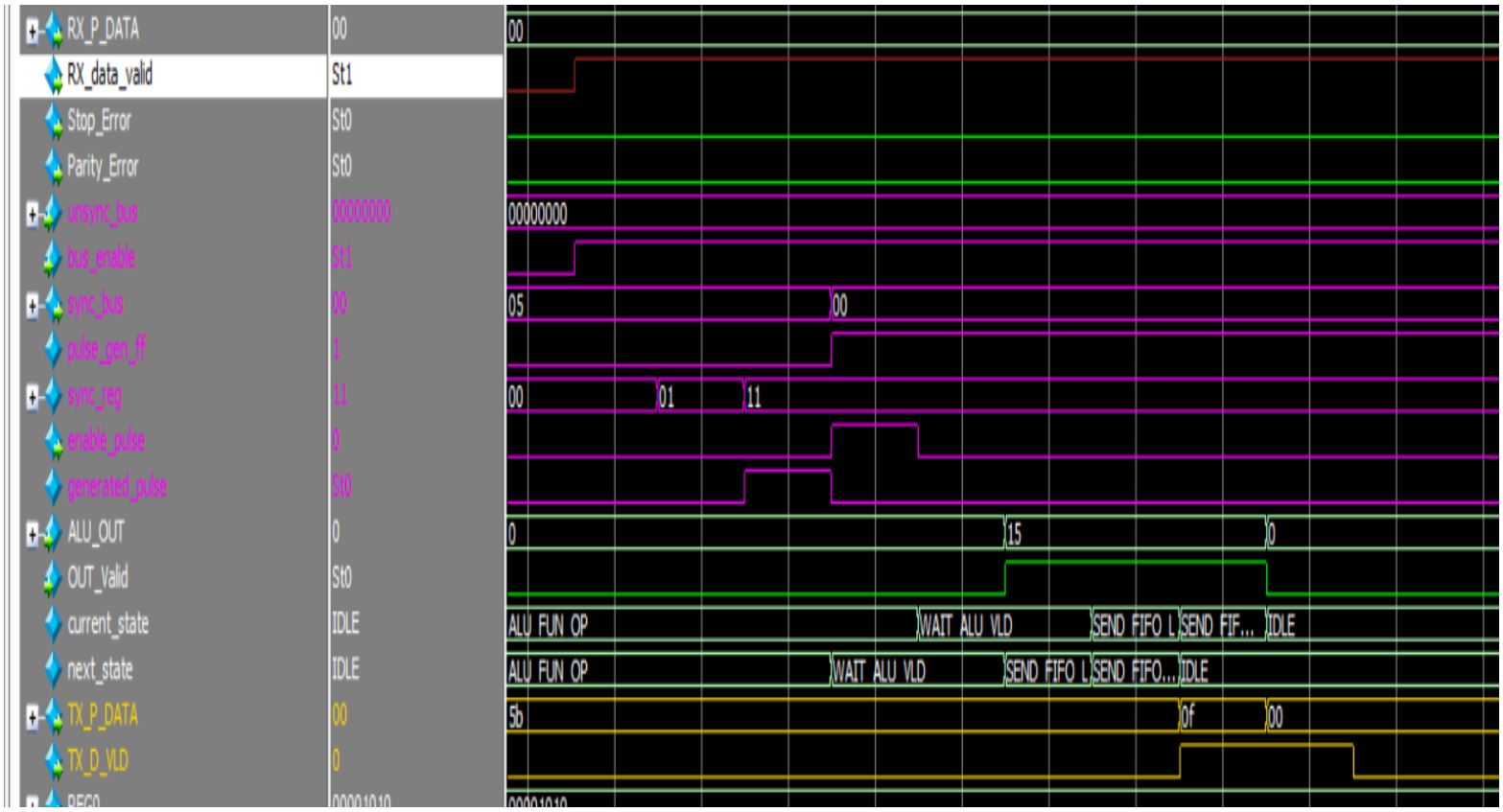
1-ADD: OpA = 10, OpB = 5, FUN = 0 -> Result = 15

alu_w_op_cmd_sequence(8'h0A, 8'h05, 8'h00, 16'h000F);



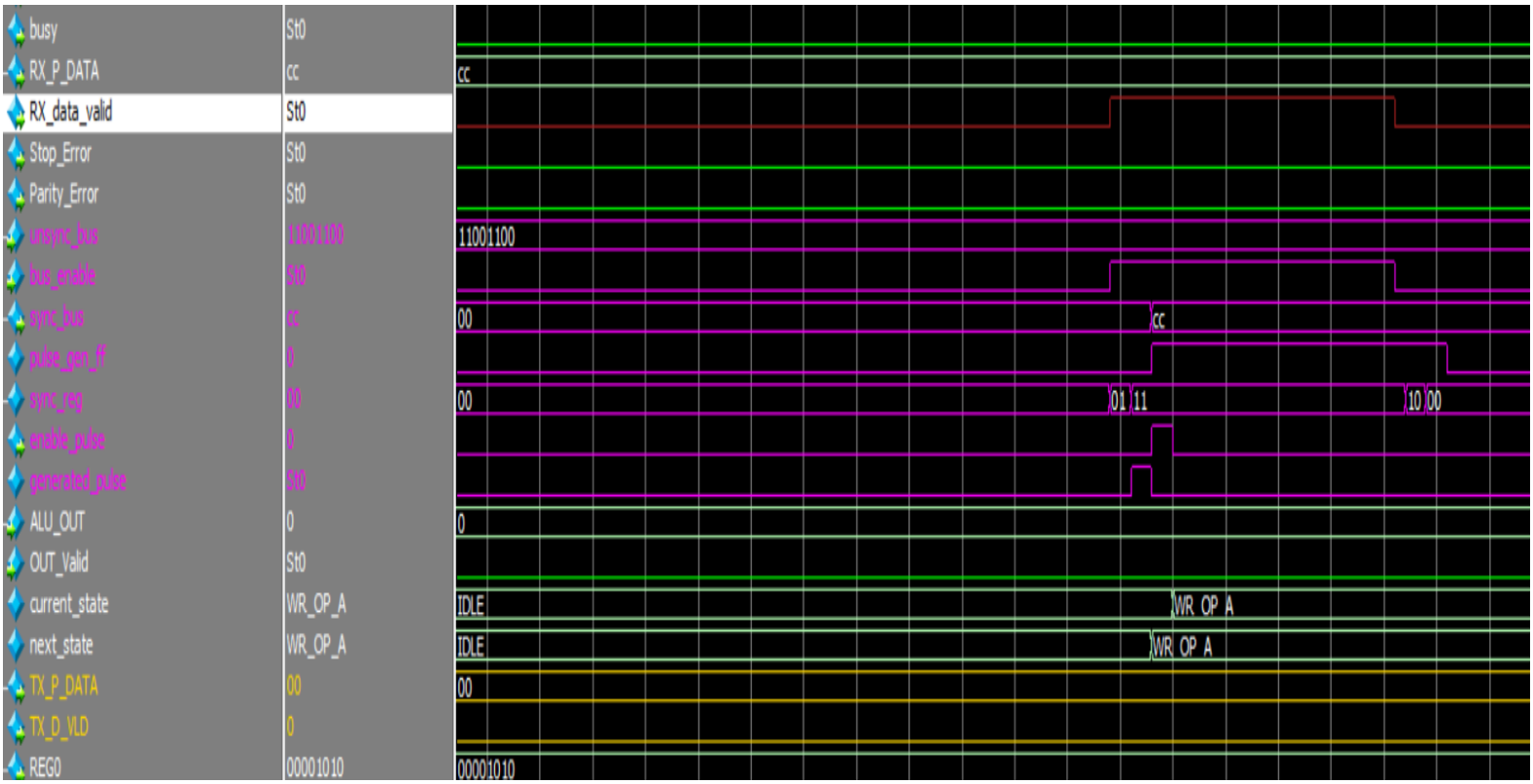


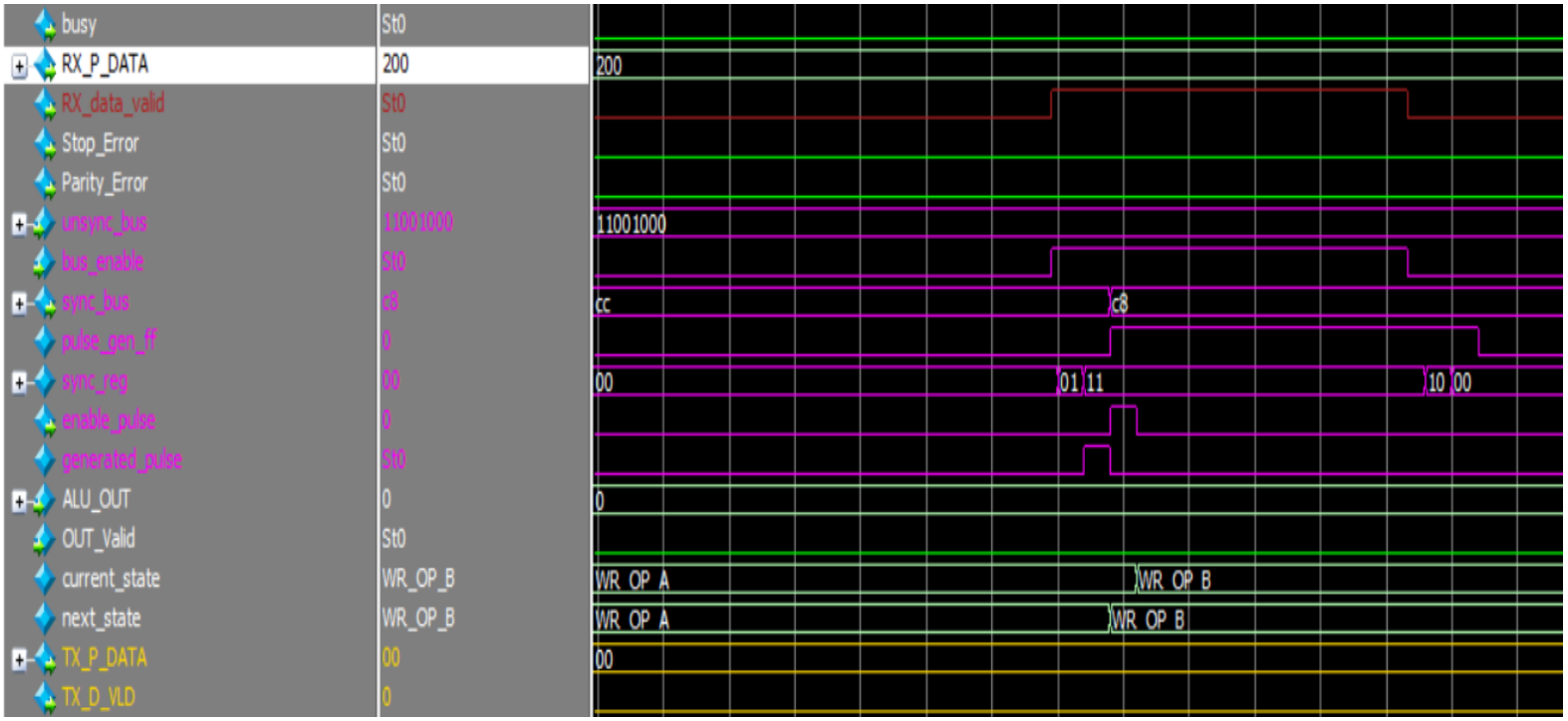

TX_OUT St1
busy St0
RX_P_DATA 05
RX_data_valid St0
Stop_Error St0
Parity_Error St0
unsync_bus 00000101
bus_enable St0
sync_bus 05
pulse_gen_ff 0
sync_reg 00
enable_pulse 0
generated_pulse St0
ALU_OUT 0
OUT_Valid St0
current_state ALU_FUN_OP
next_state ALU_FUN_OP
TX_P_DATA 5b
TX_D_VLD 0
REG0 00001010
REG1 00000101
REG2 10000001
REG3 00100000
reg_file 00000000 00000000 00...
05
00000101
0a
05
00
01
11
10
00
0
WR_OP_B
ALU_FUN_OP
WR_OP_B
ALU_FUN_OP
5b
00001010
00000000
00000101
10000001
00100000

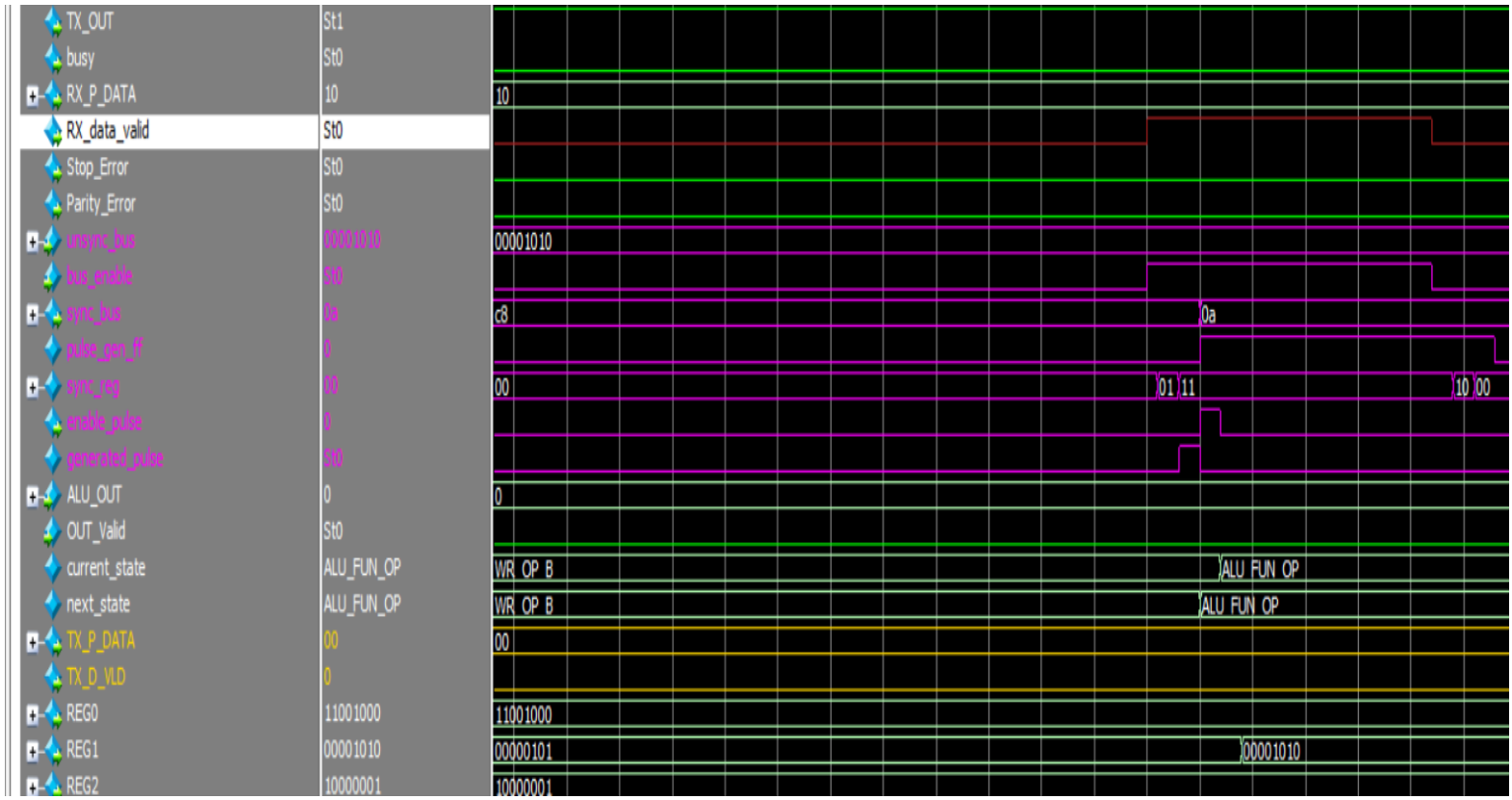

RX_P_DATA 00
RX_data_valid St1
Stop_Error St0
Parity_Error St0
unsync_bus 00000000
bus_enable St1
sync_bus 00
pulse_gen_ff 1
sync_reg 11
enable_pulse 0
generated_pulse St0
ALU_OUT 0
OUT_Valid St0
current_state IDLE
next_state IDLE
TX_P_DATA 00
TX_D_VLD 0
00
00000000
05
00
00
01
11
0
15
0
ALU_FUN_OP
WAIT_ALU_VLD
SEND_FIFO_L
SEND_FIF...
IDLE
ALU_FUN_OP
WAIT_ALU_VLD
SEND_FIFO_L
SEND_FIFO...
IDLE
5b
0f
00

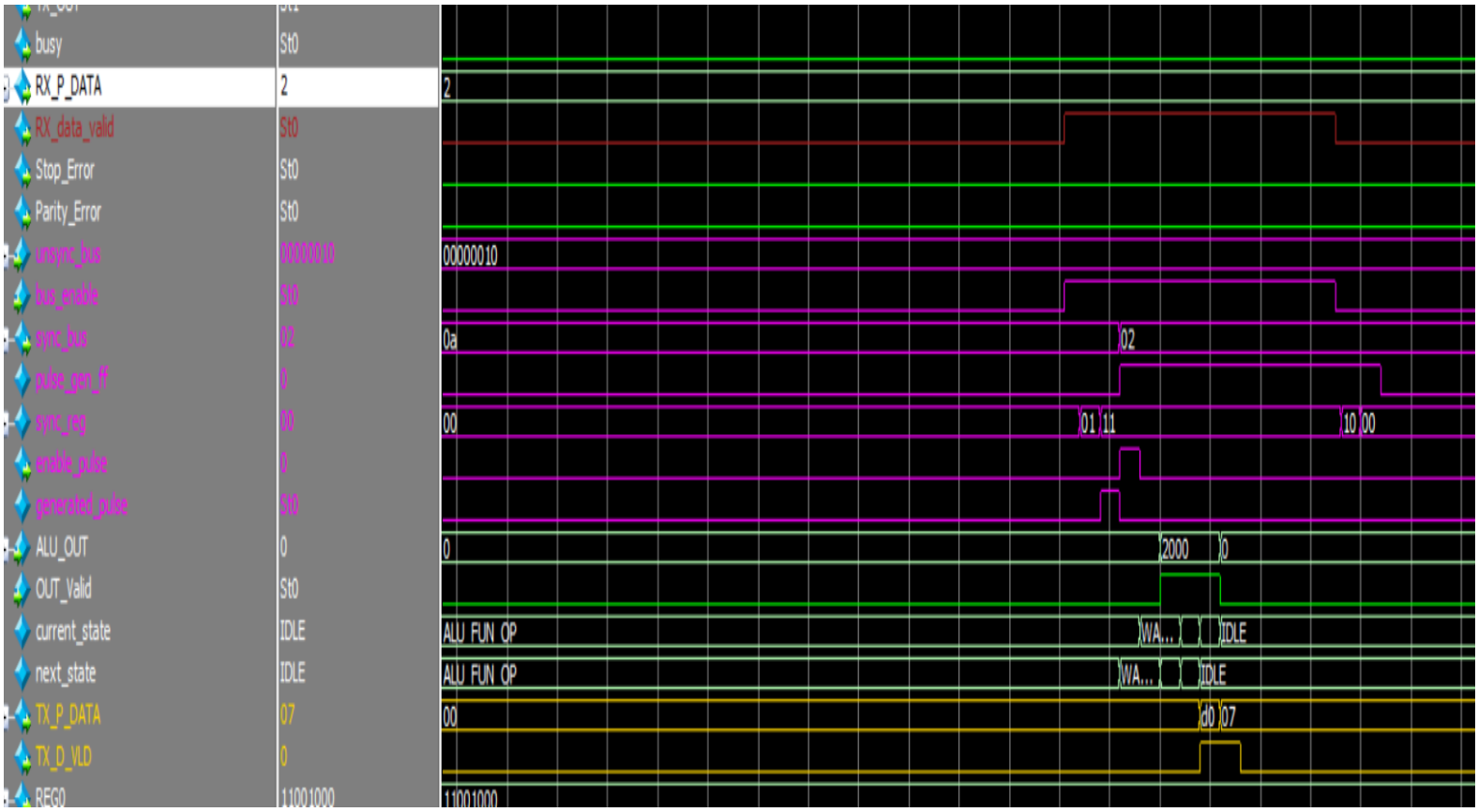
// MUL: OpA = 200, OpB = 10, FUN = 2 -> Result = 2000

alu_w_op_cmd_sequence(8'd200, 8'd10, 8'd2, 16'h07D0);



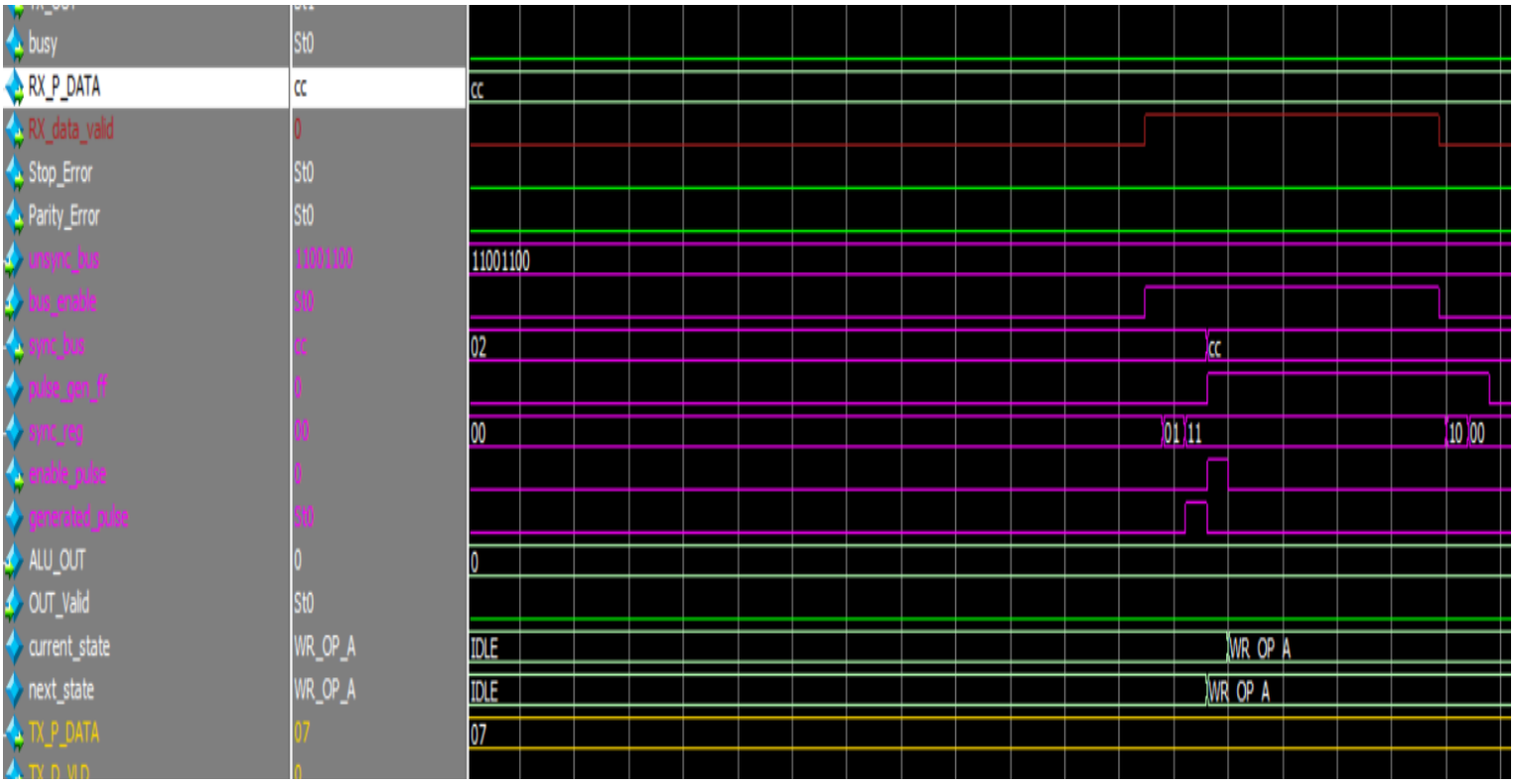


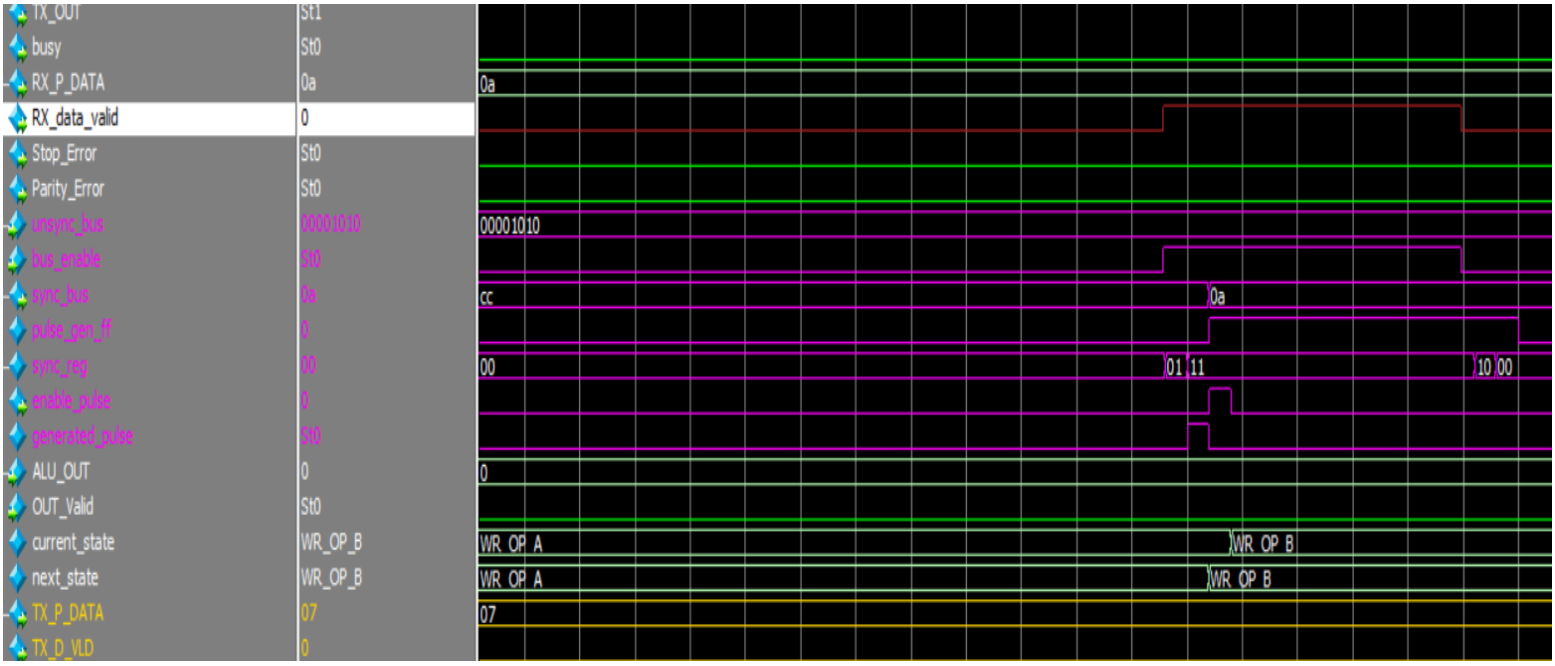

TX_OUT
St1
busy
St0
RX_P_DATA
10
RX_data_valid
St0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00001010
bus_enable
St0
sync_bus
0a
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
ALU_FUN_OP
next_state
ALU_FUN_OP
TX_P_DATA
00
TX_D_VLD
0
REG0
11001000
REG1
00000101
REG2
10000001
WR_OP_B
ALU_FUN_OP
01
11
10
00
c8

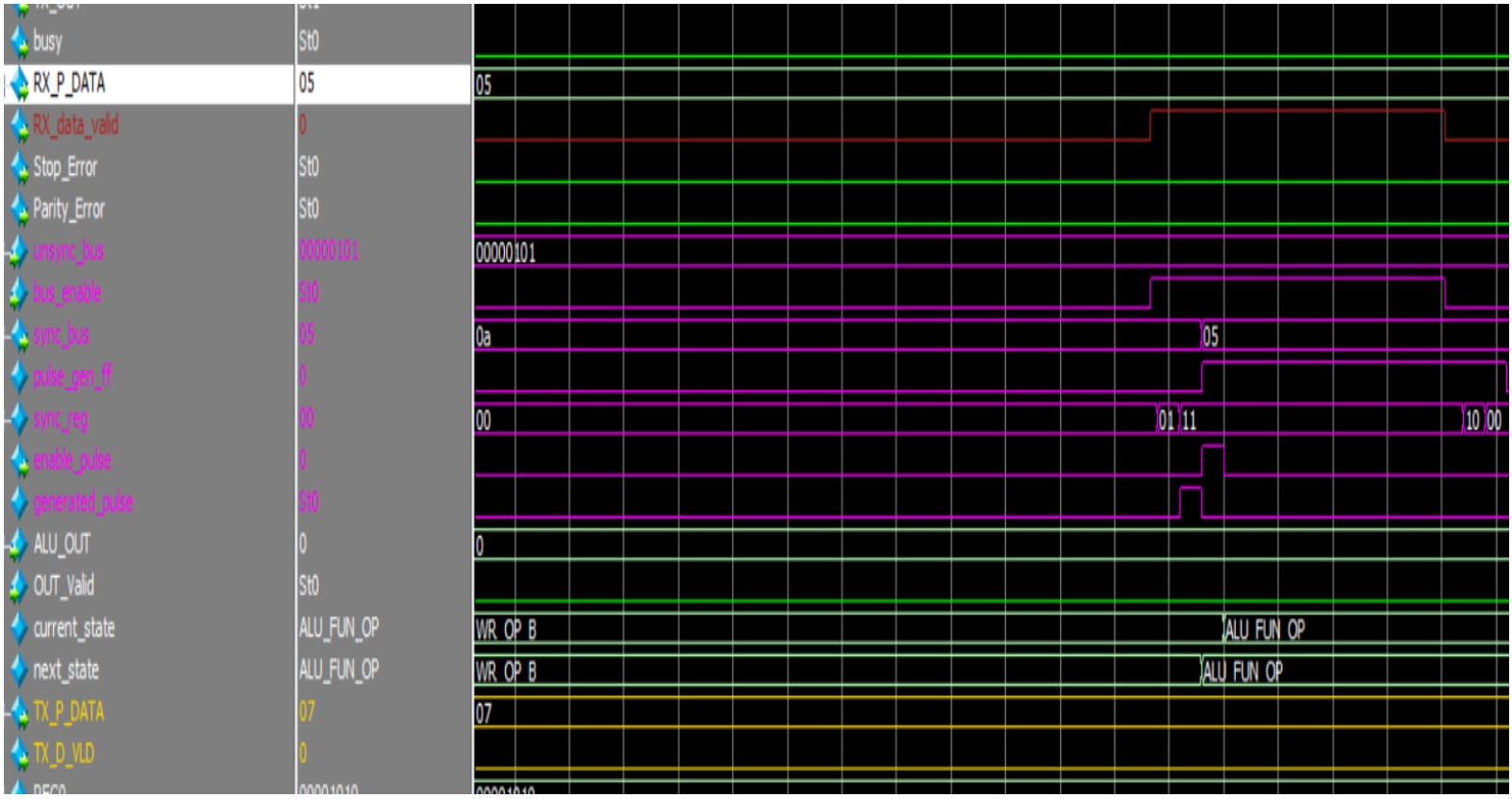

busy
St0
RX_P_DATA
2
RX_data_valid
St0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00000010
bus_enable
St0
sync_bus
02
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
IDLE
next_state
IDLE
TX_P_DATA
07
TX_D_VLD
0
REG0
11001000
0a
01
11
10
2000
ALU_FUN_OP
IDLE
d0

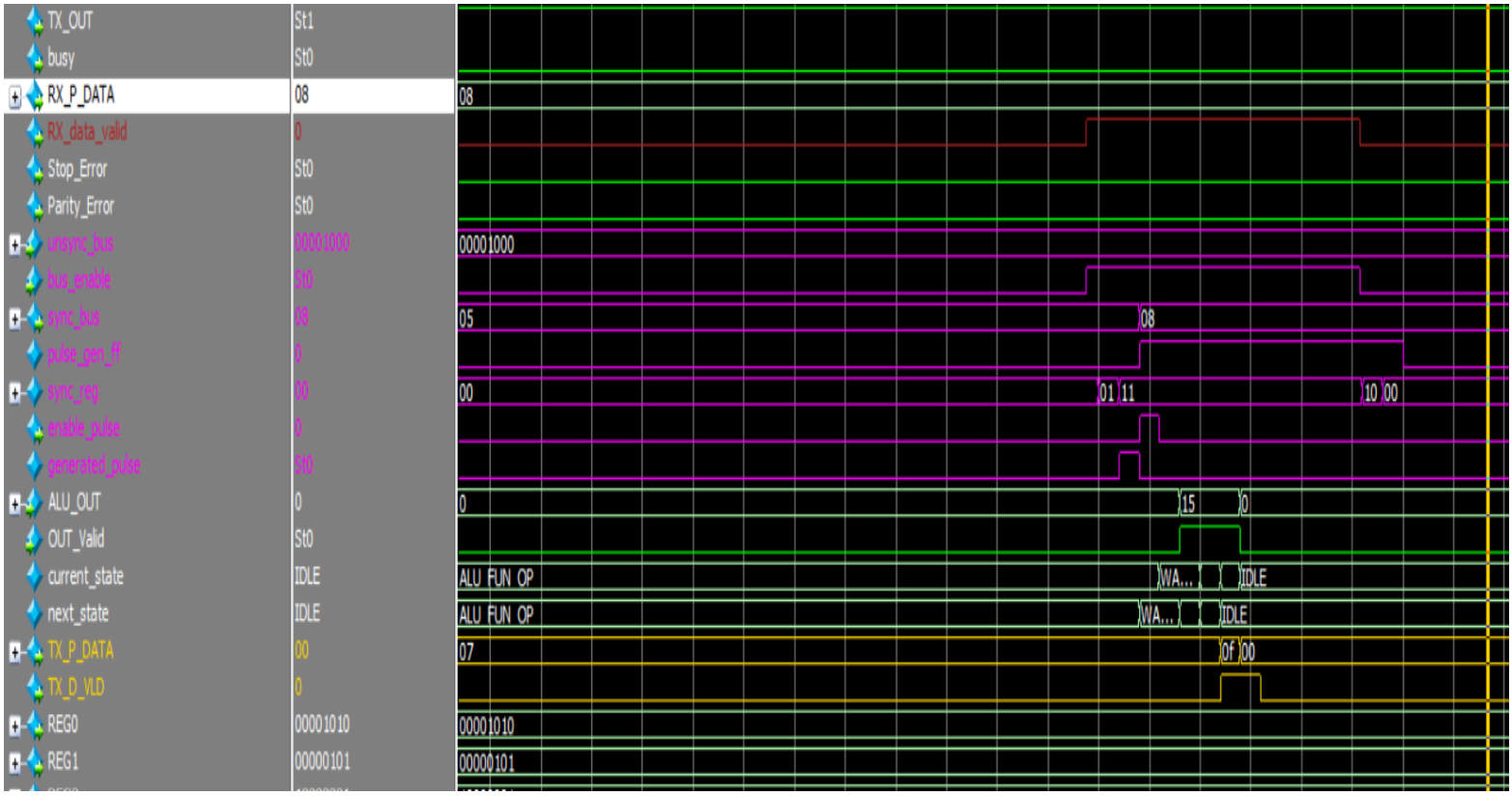
XOR OpA = 10, OpB = 5, FUN = 8 -> Result = 15

alu_w_op_cmd_sequence(8'h0A, 8'h05, 8'h08, 16'h000F);



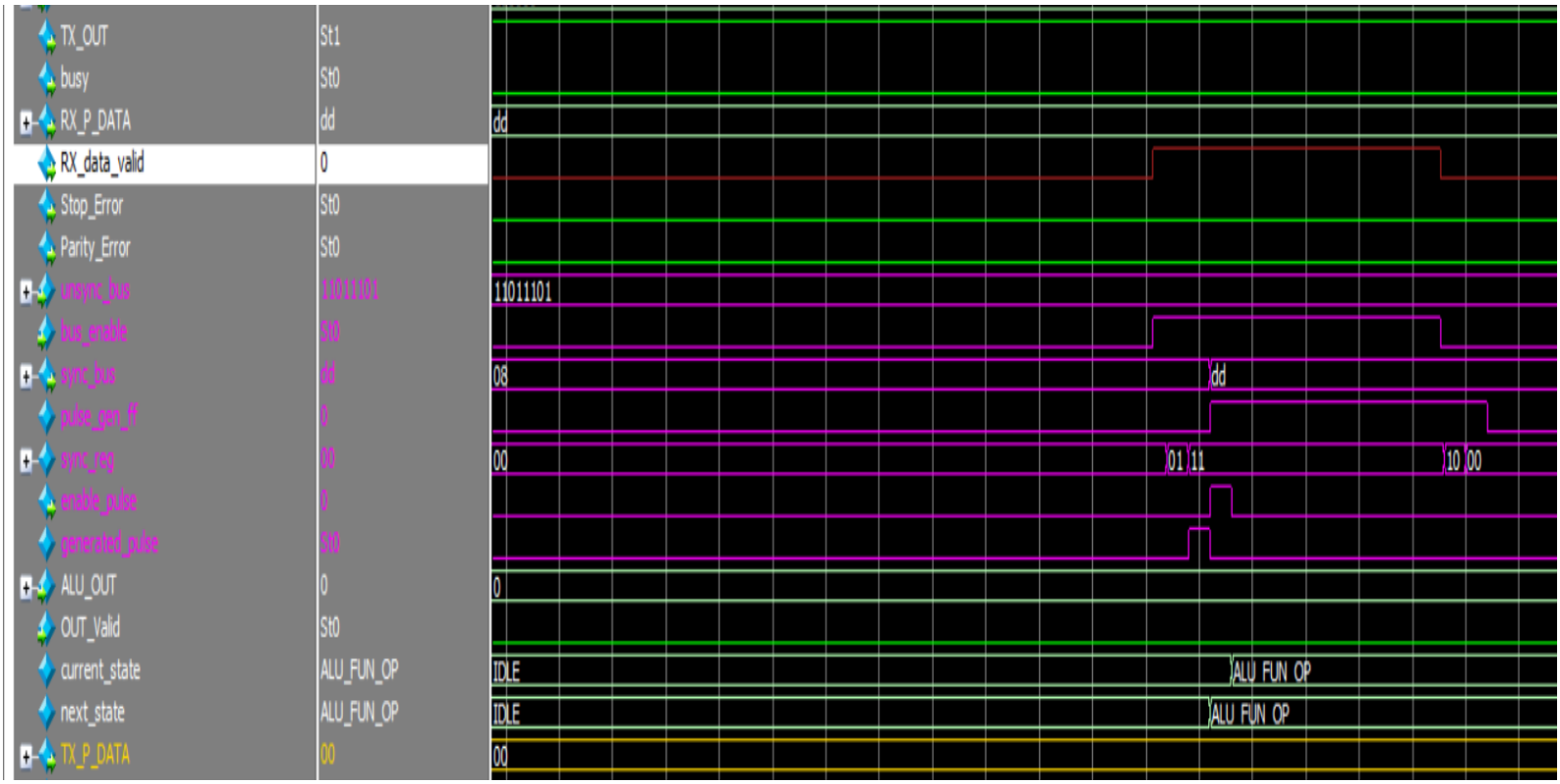


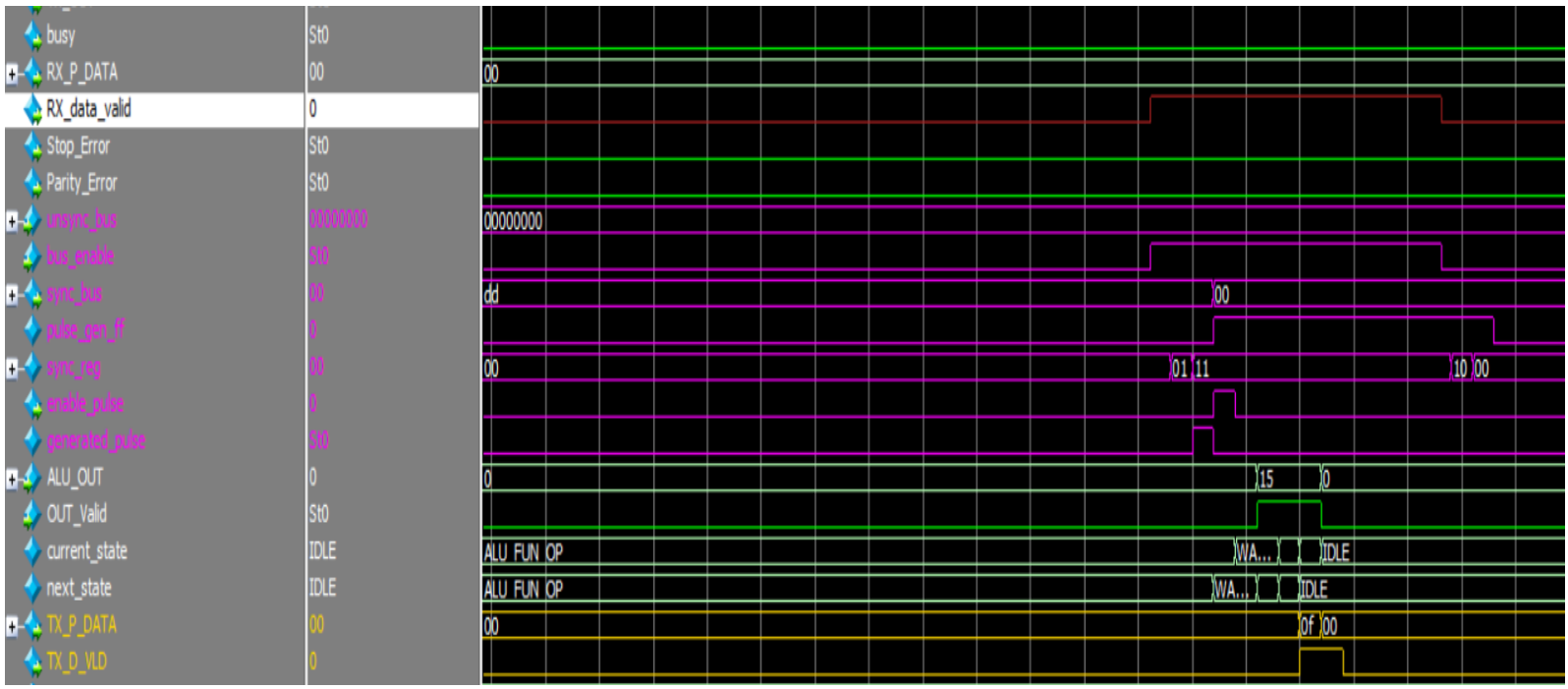

busy
St0
RX_P_DATA
05
RX_data_valid
0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00000101
bus_enable
St0
sync_bus
05
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
ALU_FUN_OP
next_state
ALU_FUN_OP
TX_P_DATA
07
TX_D_VLD
0
0a
01
11
10
WR_OP_B
ALU_FUN_OP


TX_OUT
St1
busy
St0
RX_P_DATA
08
RX_data_valid
0
Stop_Error
St0
Parity_Error
St0
unsync_bus
00001000
bus_enable
St0
sync_bus
08
pulse_gen_ff
0
sync_reg
00
enable_pulse
0
generated_pulse
St0
ALU_OUT
0
OUT_Valid
St0
current_state
IDLE
next_state
IDLE
TX_P_DATA
00
TX_D_VLD
0
REG0
00001010
REG1
00000101
05
01
11
10
15
ALU_FUN_OP
IDLE
07
0f

4: Testing ALU Operations with NO Operands

alu_w_nop_cmd_sequence(8'h00, 16'd15);





```
#           Using alternate file: ./wlfa2j9a6
#
# [WRITE REG]: Addr = 0x2, Data = 0x81
# [WRITE REG]: Addr = 0x3, Data = 0x20
# [WRITE REG]: Addr = 0x5, Data = 0xa5
# [WRITE REG]: Addr = 0xa, Data = 0x5b
# [READ REG]: Addr = 0x5
# PASS: Expected = 0xa5, Received = 0xa5
# [READ REG]: Addr = 0xa
# PASS: Expected = 0x5b, Received = 0x5b
# [ALU W_OP]: OpA = 0xa, OpB = 0x5, FUN = 0x0
# PASS: Expected = 0xf, Received = 0xf
# PASS: Expected = 0x0, Received = 0x0
# [ALU W_OP]: OpA = 0xc8, OpB = 0xa, FUN = 0x2
# PASS: Expected = 0xd0, Received = 0xd0
# PASS: Expected = 0x7, Received = 0x7
# [ALU W_OP]: OpA = 0xa, OpB = 0x5, FUN = 0x8
# PASS: Expected = 0xf, Received = 0xf
# PASS: Expected = 0x0, Received = 0x0
# [ALU W_NOP]: FUN = 0x0
# PASS: Expected = 0xf, Received = 0xf
# PASS: Expected = 0x0, Received = 0x0
# At the end of test --> Error Count = 0 | Correct Count = 10
# Break in Module sys_tb at sys_tb.v line 87
```